FACE: Fully Overlapped PD Scheduling and Multi-Level Architecture Co-Exploration on Wafer

Zheng Xu[†], Dehao Kong[†], Jiaxin Liu[†], Dingcheng Jiang[†], Xu Dai[‡], Jinyi Deng[†✉], Yang Hu[†✉] and Shouyi Yin[†‡]
[†] School of Integrated Circuits, BNRist, Tsinghua University, Beijing, China, 100084
[‡] Shanghai Artificial Intelligence Laboratory, Shanghai, China, 200433
[✉] Corresponding Authors
dengjinyi@mail.tsinghua.edu.cn, hu_yang@tsinghua.edu.cn

2026 IEEE International Symposium on High Performance Computer Architecture (HPCA) | 979-8-3315-9302-5/26/$31.00 ©2026 IEEE | DOI: 10.1109/HPCA68181.2026.11408518

***Abstract*—The rapid expansion of large language models (LLMs) parameter scales imposes unprecedented demands on compute, memory, and communication resources for inference deployment. Wafer-scale chips, leveraging advanced packaging technologies, deliver high-density integration of compute and memory with high die-to-die (D2D) communication bandwidth, providing a compelling architectural approach to satisfy these resource requirements. However, its unprecedented chip area introduces significant architectural design complexities. Wafer-scale chips feature a multi-level architecture spanning the wafer, die, and core levels, involving numerous critical design parameters and trade-offs, which still lack systematic understanding and exploration. Moreover, this poses major challenges for LLM serving scheduling. Existing methods, largely adapted from GPU-based systems, fail to fully leverage the advantages of wafer-scale chips and mitigate their limitations, making it difficult to efficiently translate massive hardware resources into actual performance gains.**

To address these challenges, we introduce FACE, a co-exploration framework for jointly optimizing multi-level architecture and serving scheduling. We first establish a flexible and extensible wafer-scale hardware template to systematically explore the optimal architecture and micro-architecture parameters. Leveraging the fine-grained control and high interconnect bandwidth of wafer-scale chips, FACE implements an LLM scheduling strategy that achieves fully overlapped prefill-decode execution and efficient KV cache management, maximizing hardware resource utilization to improve LLM service quality. Our evaluation demonstrates that FACE can achieve an average overall performance improvement of 3.68× across various LLM models and datasets compared to the state-of-the-art (SOTA) LLM serving system on wafer-scale chips.

I. Introduction

Large language models (LLMs) [6], [7], [16], [68], [69] have recently emerged as pivotal enablers of artificial intelligence (AI), powering a wide spectrum of applications including chatbots [8], [11], [57], code generation [14], [19], [65], and agent-based systems [12], [50], [64]. To effectively support increasingly complex tasks and deliver superior performance, LLMs have rapidly increased the parameter count, with recent models surpassing hundreds of billion parameters [7], [11], [16], [89]. Although this continuous growth significantly enhances their generalization capabilities, it also imposes unprecedented demands on the underlying hardware resources for the LLM inference service, including the compute, memory, and interconnect communication bandwidth.

Wafer-scale design methodology [32], leveraging advanced packaging technologies such as CoWoS [29], EMIB [21], and fan-out [70], substantially expands the available chip area and enables high-density vertical integration of critical modules—including die, power supply, cooling, and I/O interfaces. This combination of massive silicon footprint and ultra-high integration density substantially increases the compute, memory and communication resources, thereby offering a viable architectural solution to address the LLM inference demands. The successful deployment of commercial wafer-scale chips such as Cerebras WSE [45] and Tesla Dojo [66] has demonstrated the feasibility of this method. A widely adopted wafer-scale strategy is integrating NPU and DRAM chiplets together [51], [52], [66], [79], [81], [85], providing high yield and cost efficiency. Recent preliminary studies [26], [79] highlight the substantial potential of wafer-scale chips for LLM serving, achieving multi-fold performance gains over GPU cluster-based LLM serving systems.

However, existing studies [26], [79] still exhibit two significant limitations. First, they lack systematic understanding and investigation into the co-design of architecture and microarchitecture for wafer-scale chips, limiting further improvements in resource integration density and the hardware optimization for LLM workloads. Second, current LLM scheduling methods on wafer-scale chips are adapted from GPU-based approaches, lacking designs that fully leverage wafer-scale architectural advantages and mitigate limitations. Consequently, translating the vast hardware resources of wafer-scale chips into actual performance improvements efficiently remains an unresolved problem.

For the first limitation, the core challenge lies in determining the optimal configuration of compute, memory, and communication resources at different architectural levels of wafer-scale chips. At the microarchitectural level, balancing the allocation of PE arrays, vector units, SRAM, and the NoC within each core is crucial. While increasing any single resource enlarges the core area, a well-designed layout can maintain others constant within the same die area, improving area utilization. At the architectural level, the trade-off among compute die size, the number of HBM chiplets per die, and die-to-die (D2D) bandwidth must be carefully managed. Incorporating more HBM chiplets increases DRAM capacity and bandwidth,

enhancing KV cache storage and decode efficiency. However, constrained wafer area and limited I/O interfaces on compute dies result in fewer compute dies and less available I/O interfaces for D2D interconnects, thereby reducing compute power and communication bandwidth.

For the second, the main challenge in LLM inference scheduling arises from the fundamentally different characteristics of the prefill and decode phases. The prefill phase is computation-intensive, while the decode phase is memory-intensive. Thus, designing scheduling strategies to maximize hardware resource utilization in both phases is critical to improving LLM service quality. WSC-LLM [79] employs an optimized disaggregated scheduling strategy that partitions all dies on a wafer-scale chip into groups, each configured as either a prefill or decode instance, with phase-specific execution strategies. By leveraging high interconnect bandwidth of wafer-scale chips, WSC-LLM mitigates significant KV cache transfer overhead and low memory utilization that limit the disaggregated scheduling methods on GPU clusters [30], [53], [56], [91]. However, this approach still faces the following problems. First, it exacerbates the physical and topological constraints inherent to wafer-scale chips, which hinder the full exploitation of the hardware potential. Influenced by instance placement and the 2D-mesh topology, KV cache transfers from prefill to decode instances can incur significant tail latency. Architectural constraints also lead to suboptimal resource allocation ratios and instance sizes, and imbalanced decode workloads. Second, due to the relatively small computational workload of the decode phase, the compute resource utilization within decode instances remains significantly low, even when the continuous batching technique [84] is employed.

The above analysis reveals that enabling efficient LLM services on wafer-scale chips still faces substantial challenges. It requires not only careful exploration of trade-offs across multi-level architecture of wafer-scale chips but also efficient scheduling strategies to fully exploit their fine-grained control and high-bandwidth advantages while mitigating physical and topological constraints. To address these challenges, we make the following contributions:

- We propose FACE, a framework for *F*ully overlapped prefill-decode scheduling and multi-level *A*rchitecture *C*o-*E*xploration on wafer-scale chips. To the best of our knowledge, this is **the first** work to achieve it.
- We systematically analyze the design space and trade-offs across the multi-level architecture of wafer-scale chips and conduct comprehensive explorations using a highly configurable and extensible hardware template.
- We leverage the fine-grained control and high interconnect bandwidth of wafer-scale chips to achieve efficient real-time prefill-decode overlap scheduling through configuration space exploration, dynamic adaptive scheduling, and optimized memory management strategies to maximize hardware resource utilization.
- Compared to the SOTA LLM serving systems on wafer-scale chips and GPU clusters, FACE can achieve an average overall performance improvements of 3.68× and 7.23×, respectively, across various LLM models and datasets. Furthermore, FACE provides several insights into the co-design of wafer-scale architecture and microarchitecture as well as LLM scheduling.

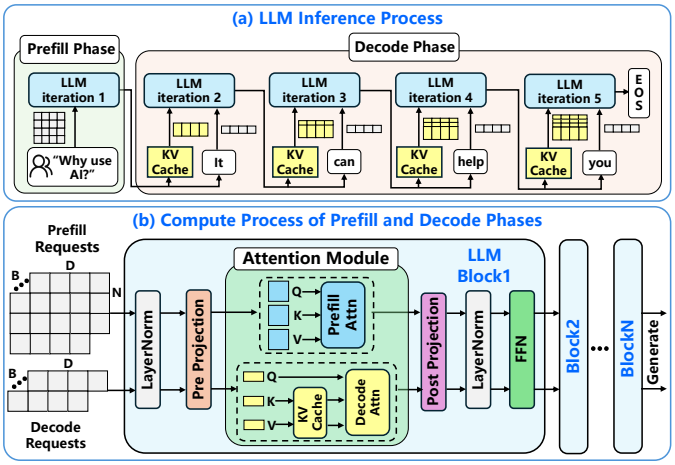

Fig. 1. LLM Inference Process.

II. Background

A. LLM Inference Process

As illustrated in Fig. 1(a), the LLM inference process consists of two phases with distinct hardware resource demands: the prefill and decode phases. The prefill phase processes the entire input sequence to generate the first token and populate the key-value (KV) cache, and is typically compute-bound [23], [58], [63], [93]. In contrast, the decode phase iteratively generates tokens using the last generated token and the KV cache, requiring high memory bandwidth and capacity [37], [40], [58], [63].

As illustrated in Fig. 1(b), the prefill and decode phases utilize the same workflow. For a single user request, the prefill input is represented as a matrix, while the decode input is a vector. Inputs sequentially traverse identical LLM blocks, each comprising linear, attention, and other operations, to generate subsequent tokens. Linear operations, such as pre-projection, post-projection, and feed-forward network (FFN), enable batching across phases to amortize weight loading costs [9]. Attention operations, computing a nonlinear weighted sum of value vectors (V) based on similarity scores between queries (Q) and keys (K), are performed separately for prefill and decode phases due to differing inputs and its non-linearity [17], [61], [83]. Other operations like LayerNorm, though non-linear, are lightweight and have minimal impact on the overall latency.

B. Wafer-scale Chip

Benefiting from advances in packaging technologies [21], [25], [29], [33], [34], [41], [42], [70], [88], wafer-scale chips have emerged as an architectural solution for achieving ultra-high integration density of hardware resources. We adopt the chiplet-based integration approach [43], [51], [62], [66], [73], [74], [79], [80], [85], [92], where compute dies are fabricated separately and subsequently integrated with interconnects and

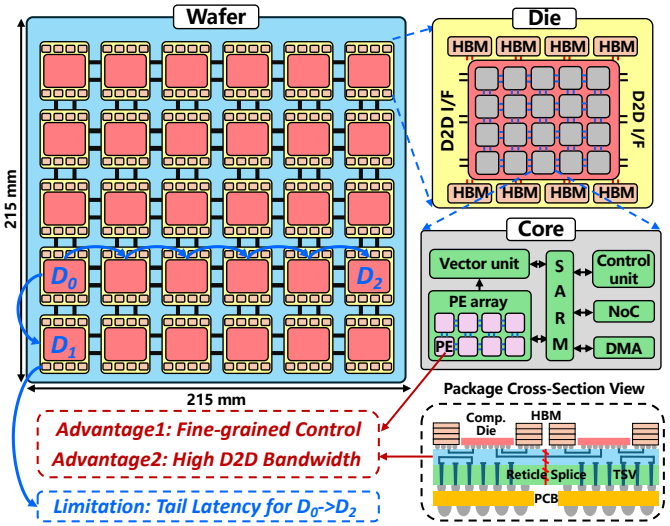

Fig. 2. Architecture of Wafer-Scale Chips.

DRAM on the wafer. Compared with monolithic integration [26], [45], [60], this approach offers higher configurability and improved manufacturing yield.

1) Hierarchical Architecture: As illustrated in Fig. 2, the architecture of wafer-scale chips is organized hierarchically into three levels: wafer, die, and core. At the wafer level, a 2D-mesh topology is employed, with Die-to-Die (D2D) interconnects enabling communication between adjacent dies. This interconnect network supports various communication patterns, including Die-to-Die, Die-to-DRAM, and DRAM-to-Die transfers. At the die level, compute and memory resources are integrated, marking a significant departure from traditional chiplet NPU designs that typically separate these functions [13], [62], [67]. Each die comprises a compute core array (compute die), on-die DRAM, a mesh-based Network-on-Chip (NoC), and D2D interfaces. At the core level, each core executes computational tasks under the core controller, which orchestrates instruction scheduling and data transfers across cores and DRAM. Each core incorporates a shared SRAM, a PE array and a vector unit for performing GEMM/GEMV and vector/scalar operations, while DMA and NoC modules ensure efficient inter-core communication.

2) Advantages and Limitations: Wafer-scale chips offer two key architectural advantages: fine-grained operation control and high D2D interconnect bandwidth. First, wafer-scale chips adopt an NPU-like [13], [15], [24], [27], [35], [38], [39], [67], [72], [77], [82] die-level architecture that supports dynamic tensor decomposition and PE-level execution control, thereby enabling fine-grained parallelism, improved pipeline overlap, and reduced computational redundancy. Second, the D2D interconnect communication, enabled by an interposer layer directly connecting compute dies and DRAMs, minimizes signal degradation and latency, achieving faster data transfer, shorter communication distances, and enhanced signal integrity. However, wafer-scale chips also face topological and physical constraints. The 2D-mesh topology causes tail latency due to varying communication hop counts, resulting in inconsistent data arrival times, while the shape of each instance is restricted by collective communication requirements.

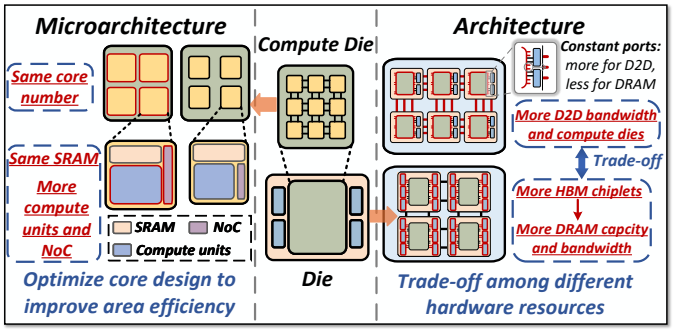

Fig. 3. Design Space of the Microarchitecture and Architecture.

C. LLM Service System

Based on continuous batching techniques [84], numerous LLM service designs for GPU clusters have been proposed, broadly classified into unified systems and disaggregated systems. Unified systems [9], [28], [40], [90] handle both the prefill and decode phases of batched requests within a single GPU node. However, due to their fundamentally different characteristics, such systems suffer from prefill-decode interference. In contrast, disaggregated systems [30], [31], [49], [53], [56], [91] decouple the prefill and decode phases, executing them on separate GPU nodes to avoid interference. However, it introduces substantial KV cache communication overhead and suffers from low memory utilization on prefill instances and poor compute utilization on decode instances.

Wafer-scale chips represent a promising new hardware paradigm, yet research on LLM service systems tailored for such architectures remains limited. WSC-LLM [79] implements disaggregated scheduling on wafer-scale chips and leverages the high D2D interconnect bandwidth to mask KV cache transfer overhead and improve memory utilization on prefill instances. However, it amplifies the physical and topological constraints inherent to wafer-scale architectures and still fails to address the underutilization of compute resources on decode instances. Therefore, there remains a significant research gap in developing efficient LLM service scheduling specifically optimized for wafer-scale chips.

III. Motivation

A. Design Space of the Hierarchical Architecture

The wafer-scale chip design space can be divided into architecture (die and wafer) and microarchitecture (core and die) levels. We aim to optimize the configuration of compute, memory, and communication resources under silicon area constraints to maximize integration density while balancing trade-offs among different hardware resources.

In microarchitecture design, under the fixed compute die area, optimal configurations of core-internal PE arrays and vector units (compute), SRAM (memory), and NoC (communication) must be carefully considered. Increasing one resource enlarges core size but may not reduce others, as resource integration also depends on core array layout. As shown in the left of Fig. 3, enhancing compute and NoC resources

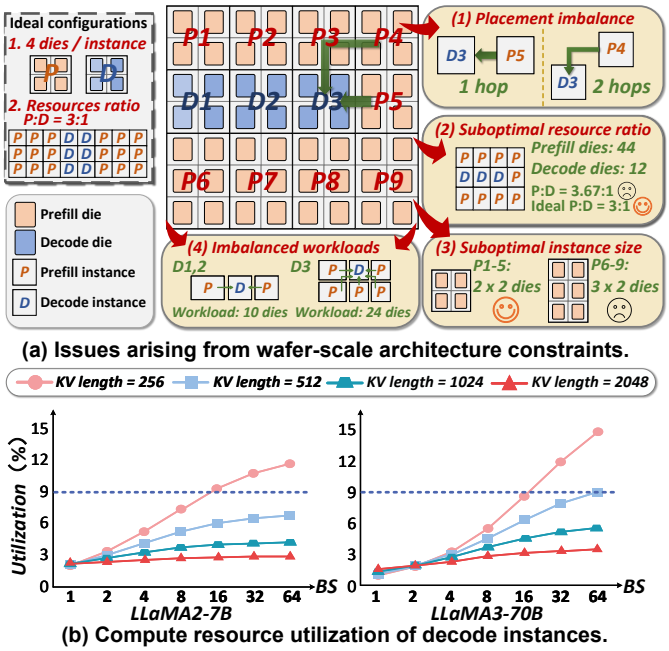

Fig. 4. Limitations of Optimized Diaggregated Scheduling on Wafer-Scale Chips.

within a core can maintain core count, improving die area utilization without compromising SRAM. Therefore, a well-designed microarchitecture is critical for maximizing hardware resource integration.

At the architectural level, under the 12-inch wafer area constraint [32], trade-offs among compute dies, HBM chiplet count, and D2D communication bandwidth must be carefully considered. As illustrated in the right of Fig. 3, integrating more HBM chiplets per die increases both memory capacity and DRAM bandwidth, thereby enhancing KV cache storage and decode efficiency. However, it consumes limited I/O interfaces on the compute die, reducing those available for D2D interconnect and thus impacting inter-die communication bandwidth, while also decreasing compute resources by occupying more die area. Furthermore, selecting an appropriate compute die size to optimize the spatial organization of compute, memory, and communication resources remains essential for achieving high-density integration.

These design factors and trade-offs highlight the importance of a co-design approach that jointly considers architecture and microarchitecture, aiming to achieve dense hardware integration and optimal performance for LLM services.

B. Limitations of Optimized Disaggregated Scheduling

Although WSC-LLM [79] successfully implements disaggregated scheduling on wafer-scale chips and takes advantage of high interconnect bandwidth to mitigate some key limitations in GPU-based disaggregated serving systems, it still faces several problems.

First, disaggregated scheduling exacerbates the topological and physical constraints inherent to wafer-scale chips. WSC-LLM [79] attempts to determine optimal instance sizes for the prefill and decode phases and allocates resources based on traffic balance between the two phases. However, as illustrated in Fig. 4(a), the following four issues arise due to the architectural limitations: (1) Guided by the resource ratio, certain prefill instances may not be placed adjacent to decode instances, resulting in significant tail latency. (2) The actual hardware resource allocation may deviate from the optimal ratio due to constraints on the total number of dies and feasible instance sizes. (3) Due to constraints imposed by layout and instance geometry, certain instance sizes deviate from the optimal configuration. (4) Since each prefill instance transmits the KV cache only to its nearest decode instance, this can lead to imbalanced decode workloads.

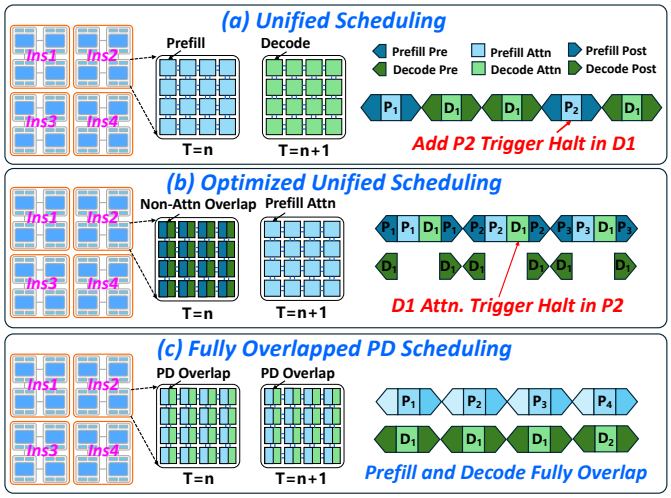

Fig. 5. New Scheduling Opportunities for Enhancing LLM Service Quality on Wafer-Scale Chips.

Second, the compute resource utilization of the decode instances is extremely low. Fig. 4(b) presents the compute utilization of decode instances under various batch sizes and KV lengths for both LLaMA2-7B and LLaMA3-70B models [22], [69]. All configurations—including hardware settings, instance sizes, and parallel strategy—follow the default setup in WSC-LLM [79]. It can be observed that, in the vast majority of cases, the compute resource utilization of decode instances is below 9%, leading to resource inefficiency.

These constraints collectively hinder the full exploitation of wafer-scale architectures for LLM services under disaggregated scheduling, highlighting the need for new LLM scheduling strategies specifically designed for the unique wafer-scale characteristics.

C. New Scheduling Opportunities and Challenges

Disaggregated scheduling amplifies the physical and topological constraints inherent to wafer-scale chips. Therefore, we aim to enable concurrent execution of the prefill and decode phases within a single instance. However, directly applying the unified scheduling strategy [40], [90] described in Section II-C is inefficient on wafer-scale chips, as it inherently serializes the execution of the prefill and decode phases, leading to prefill-decode interference, as illustrated in Fig. 5(a). While certain optimizations [9], [28] support the parallel execution of linear operators from both phases, the attention operator—accounting for a dominant portion of overall execution time, especially with longer input sequences [37]—remains

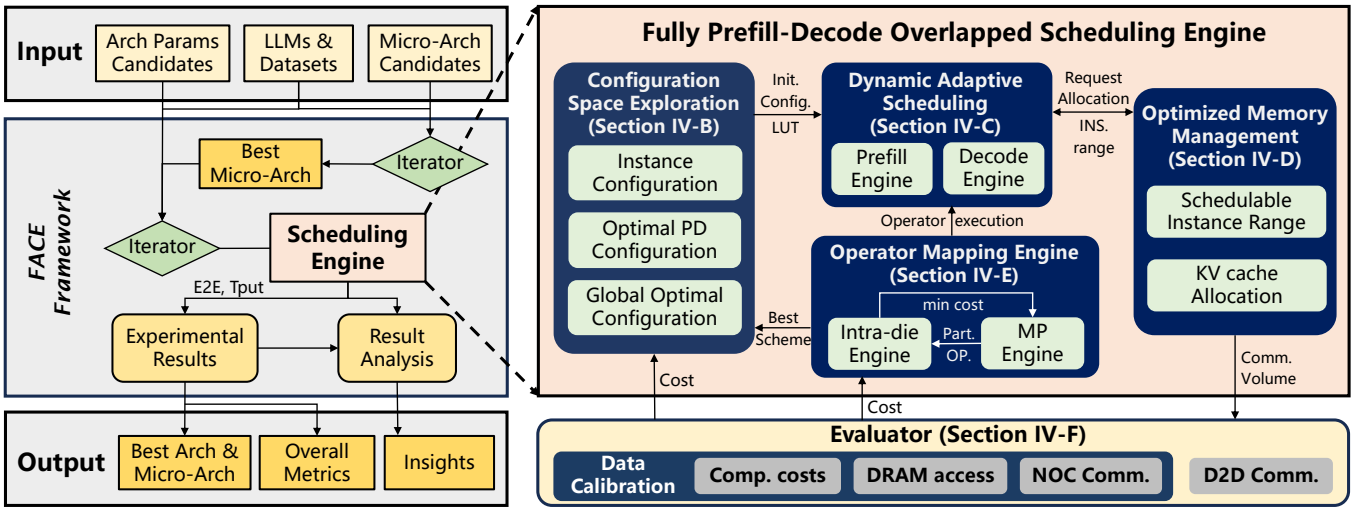

Fig. 6. FACE Framework.

serialized, as shown in Fig. 5(b). Consequently, the prefill-decode interference persists.

To address this problem, we propose enabling parallel execution of the attention operations from both phases, thereby fundamentally eliminating prefill-decode interference and maximizing resource utilization. The fine-grained control of wafer-scale chips provides the foundation for this approach. Through control I/O, the host can directly manage the controller and DMA engine of each core to precisely regulate the sizes of prefill and decode tiles assigned according to scheduling requirements, thereby achieving fully overlapped prefill–decode execution, as shown in Fig. 5(c).

However, achieving fully overlapped prefill-decode execution on wafer-scale chips poses significant challenges. ***Challenge 1: Concurrent attention computation requires comprehensive configuration exploration and fine-grained pipeline orchestration.*** Factors such as prefill token counts, decode batch sizes, and attention tile sizes all influence the efficiency of concurrent execution, while considering constraints like SRAM capacity. ***Challenge 2: Online LLM services demand extremely real-time responsiveness.*** Thus, it is crucial to design a real-time scheduling mechanism that leverages pre-explored configurations for concurrent attention execution. ***Challenge 3: Dynamic nature of LLM workloads introduces further complexity.*** Since requests may vary significantly in input sequence lengths, it becomes challenging to maintain high hardware utilization while adapting to these variations.

IV. FACE Framework

A. FACE Overview

FACE is a prefill-decode overlapped scheduling and multi-level architectural design specifically for wafer-scale systems, pioneering research in this domain. As illustrated in the left of Fig. 6, FACE takes as input the candidate parameters for both architecture and microarchitecture, along with LLM models and test datasets (workloads).

For multi-level architecture exploration, FACE adopts a two-stage approach to optimize hardware parameters of the wafer-scale chip. First, under fixed compute die area constraints, microarchitecture configurations are evaluated by measuring LLM block execution times within an instance, identifying the configuration with the shortest time as optimal. Subsequently, the optimal microarchitectural configuration is paired with various architectural parameters, leveraging the scheduling engine for a comprehensive exploration. The configuration that delivers the highest LLM service quality is identified as the optimal wafer-scale architecture. The parameter selections are detailed in Section V-A.

For the scheduling engine, which is on the host to manage task scheduling on the wafer-scale chip, we integrate three key strategies to address the challenges posed by achieving fully overlapped prefill-decode scheduling: *Configuration Space Exploration (CSE)* for *Challenge 1*, *Dynamic Adaptive Scheduling (DAS)* for *Challenge 2*, and *Optimized Memory Management (OMM)* for *Challenge 3*. CSE leverages the fine-grained operation of wafer-scale chips to pre-explore viable configuration spaces that support concurrent execution of attention operations. DAS allocates user requests in real time based on the guidance provided by CSE. Notably, CSE is performed offline, incurring no additional system overhead during the runtime of DAS, while its generated configuration files are stored on the host, thus without any on-wafer storage consumption. Furthermore, OMM leverages the high D2D bandwidth of wafer-scale chips to expand the schedulable instance space and optimize KV cache storage, thereby improving adaptability to dynamic LLM workloads.

B. Configuration Space Exploration

Configuration space exploration (CSE) tackles three key problems: (i) determining the optimal instance arrangement and sizes under fixed wafer-scale architecture; (ii) identifying configurations for complete prefill-decode execution overlap within each instance, including the allocation ratio of prefill and decode requests and tile sizes for two-phase attention operators; and (iii) ensuring pre-explored configurations guide real-time LLM scheduling effectively. To tackle these, we

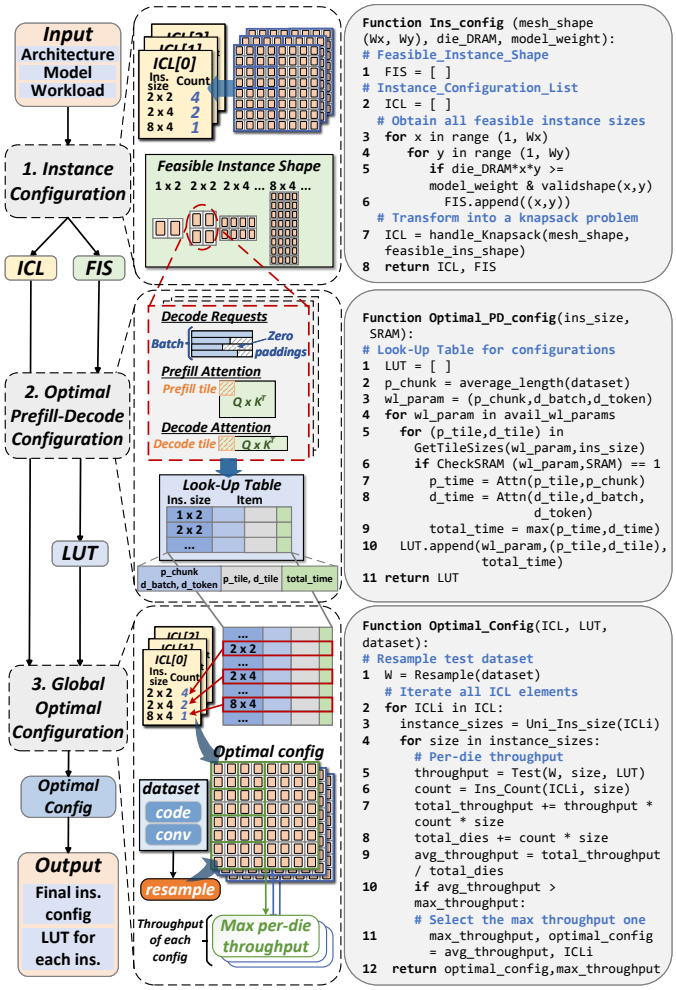

Fig. 7. Configuration Space Exploration Process.

propose instance configuration, optimal prefill-decode configuration, and global optimal configuration strategies. By inputting wafer-scale chip architectural parameters, models, and workloads, CSE can generate an optimal global instance layout (for problem i) and a look-up table (LUT) that guides the actual execution at each instance (for problem ii and iii), thereby facilitating efficient LLM inference.

1) Instance Configuration: Given the hardware parameters of the wafer-scale chip, the objective of instance configuration is to derive the complete set of feasible instance shapes (FIS) and the corresponding instance configuration list (ICL), thereby facilitating subsequent optimization.

FIS comprises all viable instance shapes meeting two key constraints: (i) sufficient DRAM capacity to store one full copy of model weights per instance, due to model parallelism; and (ii) topological ring closure (e.g., 2×2, 2×3, etc.) to ensure efficient intra-instance communication via the bidirectional ring algorithm (detailed in Section IV-E) on the 2D-mesh interconnect network of wafer-scale chips. Once the FIS is established, we formulate the generation of ICL as a variant of the knapsack problem, considering the total die count and global topological constraints. Each ICL element specifies a unique combination of different instance shapes and their counts. For example, as illustrated in the top of Fig. 7, ICL[0] contains four 2×2, two 2×4, and one 8×4 instances, which fill the entire 8×8 die array of the wafer-scale chip.

2) Optimal Prefill-Decode Configuration: The objective of the optimal prefill-decode configuration is to construct a look-up table (LUT) offline and in advance under the given wafer-scale microarchitecture to guide the subsequent dynamic adaptive runtime scheduling (Section IV-C). As shown in the middle of Fig. 7, the LUT comprises three parts: the lookup block, the execution block, and the result block. The lookup block contains workload parameters including the prefill chunk size, the decode batch size, and the decode token count. The execution block records tile sizes for the prefill and decode phases, while the result block stores the corresponding attention execution time.

To ensure that the LUT can effectively support runtime scheduling, we apply several optimizations in handling prefill and decode requests to reduce the number of workload parameters in the lookup block. For prefill requests, we employ the chunked prefill technique [9], setting the chunk size to the average input token length from the test dataset and processing one chunk per iteration, which eliminates the need to consider varying prefill input sequence lengths and batch sizes, thus reducing the exploration complexity. For decode requests, we use zero-padding to align varying token lengths within a batch to the longest token, adopting this length as the batched decode token count to enable efficient subsequent processing.

We achieve prefill-decode overlapping execution by optimizing attention computation configurations for both phases. Specifically, we explore the configuration space by traversing the decode batch size, decode token count, and attention operator tile sizes. For each configuration, prefill and decode attention tiles are jointly mapped to compute cores within the instance and must respect the SRAM capacity constraint of the core. If the configuration satisfies the constraint, we then measure the execution latency of the concurrent execution of prefill and decode attention operations. The measured latency, along with its associated configuration, is subsequently recorded in the LUT. It is worth noting that for each unique combination of workload parameters in the lookup block, the LUT retains only one configuration of prefill and decode tile sizes that yields the shortest execution time.

3) Global Optimal Configuration: The global optimal configuration leverages the previously constructed ICL and LUT to determine the optimal global instance layout for initializing the basic setup in runtime scheduling.

As illustrated in the bottom of Fig. 7, we iterate over all elements in the ICL. For each element ICL[i], we identify all the distinct instance sizes involved. For each instance type, we utilize the LUT to guide execution in order to achieve instance-level optimal per-die throughput. The workload W is obtained by sampling the evaluation dataset. This is because the macroscopic workload distribution—the ratio of different request lengths over longer time scales—remains relatively stable, enabling reliable estimation with only a small number of samples [44], [78], [87]. Furthermore, DAS (Section IV-C)

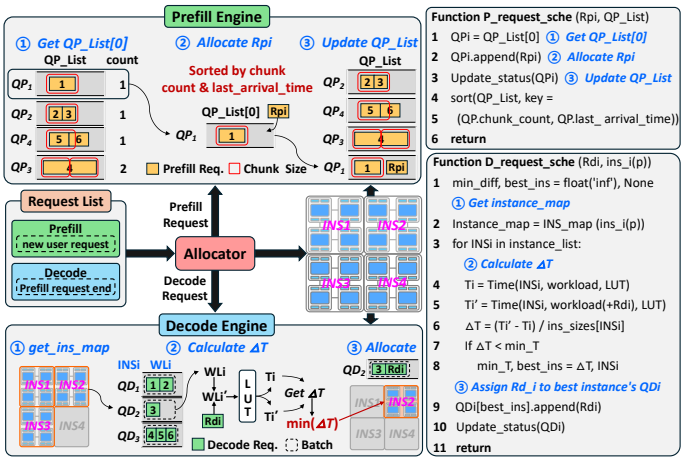

Fig. 8. Dynamic Adaptive Scheduling Process.

dynamically balances workload distribution across instances, allowing FACE to effectively handle short-term bursty workloads that may not be captured in the sampled distribution. Each instance type is evaluated by executing workload W, yielding its corresponding per-die throughput. Finally, we aggregate the results of all instances to estimate the global average per-die throughput of ICL[i], and select the configuration that achieves the highest global average throughput across all ICL elements as the optimal configuration.

C. Dynamic Adaptive Scheduling

Based on CSE, we develop a dynamic adaptive scheduling (DAS) mechanism to address the highly dynamic and real-time workload demands of online LLM service systems. DAS is a software-based adaptive scheduling algorithm that can dynamically adjust scheduling decisions based on runtime workload characteristics, ensuring optimal prefill–decode overlapped execution all the time to maximize overall system performance and maintain low-latency responses.

Although FACE enables concurrent execution of prefill and decode requests within a single instance, their distinct computational characteristics necessitate differentiated management. Accordingly, we utilize the prefill engine and the decode engine to independently process user requests corresponding to their respective phases. Within each instance ins_i, the prefill engine and the decode engine maintain a queue QP_i and QD_i respectively to manage user requests, ensuring efficient and organized request processing.

1) Prefill Engine: We sort all QP_i based on the following two criteria and insert them into a global list QP_List accordingly: (i) the number of remaining chunks to be processed in QP_i, with the chunk size aligned with the setting adopted in configuration space exploration (CSE); and (ii) the arrival time of the most recently enqueued request in QP_i. The first criterion takes precedence over the second. As shown in the top of Fig. 8, the initial queues are ordered according to the aforementioned criteria, resulting in the following sequence: QP_1, QP_2, QP_4, and QP_3. Upon receiving a new prefill request, we assign it to the first queue in the sorted QP_List and update the list to facilitate efficient assignment for subsequent requests. Within each queue QP_i, requests are processed in a First-Come-First-Serve (FCFS) manner.

The complexity of the prefill engine's queuing algorithm is $O(n)$, where n denotes the number of instances. In practice, n is typically very small—usually only a few to a dozen instances—making the decision-making process highly efficient and ensuring that it does not significantly hinder the execution of FACE.

2) Decode Engine: Our goal is to schedule newly generated decode requests in a manner that drives the runtime state of each QD_i as close as possible to the optimal configuration explored in CSE. Notably, due to heterogeneous runtime situations across instances, a request completed prefill phase in QP_i may not necessarily proceed to decode in QD_i. Leveraging the high D2D bandwidth of wafer-scale chips, we can allocate the decode request to a more urgently needed instance.

As shown in the bottom of Fig. 8, for a new decode request Rd_i , we first determine its schedulable instance range $Instance\_map$ based on the instance that executed its prefill phase and the relevant architectural constraints (detailed in Section IV-D1). Then, we iterate through all instances in $Instance\_map$, using the LUT to estimate the current per-iteration execution time T_i of each instance for workload W_L. Given the dynamic nature of LLM workloads, an exact matching entry for the current workload configuration may not always exist in the LUT. To handle this, we adopt a match-and-evaluate strategy, which sequentially scans the lookup block in the LUT according to the actual workload. For each workload, we identify entries that match both the prefill chunk size and the decode batch size, and then select the one with the minimum Euclidean distance as the closest configuration, using its recorded latency as the estimated T_i. Based on the obtained T_i, we further estimate the execution time T_i' of the updated workload W_L' after incorporating Rd_i, and compute the corresponding incremental latency $\Delta T = T_i' - T_i$. Finally, Rd_i is assigned to the instance with the smallest ΔT.

The algorithm of the decode engine has a complexity of $O(n')$, where n' denotes the number of instances within the schedulable range, and each iteration mainly requires two LUT queries. Thanks to the optimizations introduced in Section IV-B2, each LUT lookup typically involves only a one-dimensional Euclidean-distance computation. Consequently, the scheduling overhead of the decode engine is insignificant compared with the request execution time and therefore does not affect scheduling responsiveness.

D. Optimized Memory Management

Wafer-scale chips feature significantly higher D2D bandwidth, often exceeding that of DRAM bandwidth. This implies that, in the absence of D2D link contention, cross-die DRAM read and write operations are primarily constrained by the DRAM bandwidth itself. Leveraging this architectural advantage, we enable a more flexible scheduling space for decode requests. Building on this foundation, we develop an optimized memory management (OMM) strategy to (i) determine the feasible scheduling space of decode requests; and (ii) manage

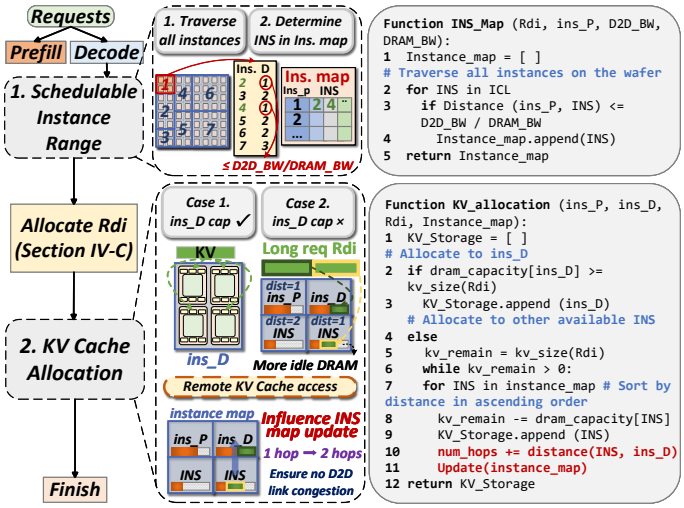

Fig. 9. Optimized Memory Management Process.

the storage of their corresponding KV caches, thereby enabling higher-quality LLM services.

1) Schedulable Instance Range: DAS assigns each decode request Rd_i to the instance within the schedulable range that minimizes the per-iteration latency increment. Our objective is to ensure that no D2D link contention occurs within the schedulable range, so that the bottleneck of inter-instance data transfer remains dominated by DRAM access, thereby avoiding any additional communication overhead. The schedulable instance range is constrained by the instance ins_p that executed the corresponding prefill phase, as well as by the available D2D and DRAM bandwidth. We define the distance between two instances ins_i and ins_j, denoted as $\mathrm{Distance}(ins_i, ins_j)$, as the number of instances along the shortest path between their geometric centers (inclusive of ins_i and exclusive of ins_j). For a decode request Rd_i that has just completed its prefill phase on ins_p, an instance ins can be included in its schedulable set if:

$$Distance(ins_p, ins) \leq \frac{D2D\_BandWidth}{DRAM\_BandWidth} \quad (1)$$

The D2D-to-DRAM bandwidth ratio defines the maximum distance for cross-instance DRAM access without D2D congestion under time-division multiplexing. As shown in the top of Fig. 9, we iterate over all instances to obtain the set $Instance\_map$, which represents the schedulable instance range for Rd_i.

For each ins_p, the algorithm to determine its schedulable instance range has a complexity of $O(n-1)$, where n denotes the total number of instances (excluding itself). This design allows DAS to efficiently refresh the schedulable range before each decision without incurring any noticeable software scheduling overhead.

2) KV Cache Allocation: Once the request Rd_i is assigned to instance ins_d for execution via the DAS strategy, we proceed to manage its KV cache storage. As illustrated in the bottom of Fig. 9, the KV cache is preferentially stored on ins_d to minimize data movement. Since the instance size has been pre-explored in Section IV-B3, ins_d is generally capable of storing the complete KV cache of Rd_i. However, in practice, if Rd_i is a burst request with an exceptionally long input token length, its KV cache may exceed the remaining DRAM capacity of ins_d. To avoid request queuing from memory exhaustion, we partially offload KV cache to other instances with available memory. The target instance for offloading is selected based on distances to ins_d and remaining DRAM capacity. After each allocation, we update DRAM capacities of all instances.

Notably, KV cache offloading across multiple instances mainly occurs for requests with exceptionally long token sequences. Since offloaded KV caches require multiple remote accesses via D2D links, the corresponding link weights are accordingly increased (from 1 hop to 2 hops), as shown in the bottom of Fig. 9. During each update of the schedulable range, these higher-weight links are considered to ensure congestion-free D2D communication.

E. Operator Mapping Engine

As discussed previously, we achieve fully overlapped prefill-decode execution through fine-tuned partitioning and scheduling of attention operators. Now, we describe the concrete implementation of attention computation on wafer-scale chips. Fig. 10 illustrates the execution of the attention operator during the prefill phase, thereby exemplifying the operational principles of our operator mapping engine. Based on the FlashAttention algorithm [17], [18], [61], [86], our approach also incorporates targeted optimizations leveraging the Network-on-Chip (NoC) interconnect. The operator mapping engine comprises two core modules: the Model Parallelism (MP) Engine and the Intra-die Engine.

1) Model Parallelism Engine: Within each instance, we leverage the MP Engine to implement efficient model parallelism across all dies, including tensor parallelism, sequence parallelism and pipeline parallelism. Specifically, we partition attention operators into sub-computation tasks based on the number of batches and attention heads or head groups (e.g., for GQA models), and map these sub-computation tasks across multiple dies. For the All-gather and All-reduce collective communication required by model parallelism, we employ a bidirectional ring algorithm [75], [79], which is highly compatible with the 2D-mesh topology of wafer-scale chips, thereby ensuring efficient data exchange.

2) Intra-die Engine: During the execution of sub-computation tasks on an individual die, these tasks are further decomposed in a fine-grained manner along the head or head-group and output sequence length dimensions into nano-computation tasks, which are then mapped to distinct core groups for parallel processing. Within a core group of size $x \times y$, the x Q slices are broadcast along the row dimension, while the y K and V slices are broadcast along the column dimension. The output of each core is propagated along the row dimension, incrementally summed to produce partial sums for the x O slices. By leveraging the Network-on-Chip (NoC) for efficient broadcast operations, this approach reduces DRAM access for K and V heads to $1/x$ of the original,

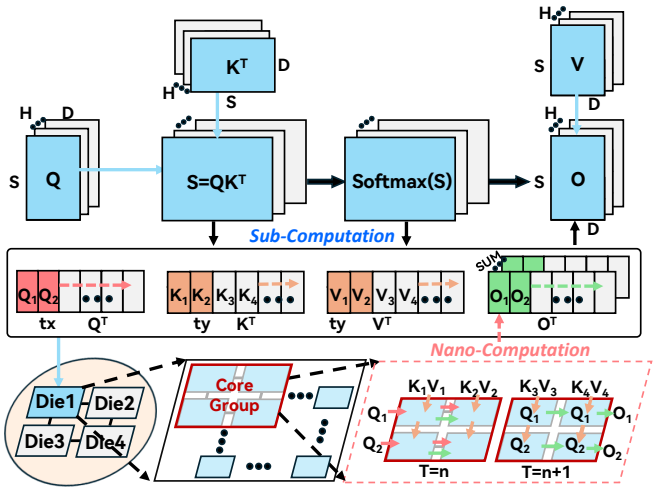

Fig. 10. The Partition and Mapping of Attention Operators.

significantly mitigating redundant DRAM access overhead and thereby enhancing overall performance.

To further enhance computational efficiency, we also introduce a dual-head pipeline strategy that enables concurrent utilization of the processing element (PE) array and vector units within each core. This optimized dataflow schedules the computation of two attention heads or head-groups simultaneously on each core group. Specifically, while the PE array performs matrix multiplication for one head or head-group, the vector unit concurrently execute data movement and softmax operations for the other head or head-group. Notably, the granularity of nano-computation tasks needs to be carefully considered so that the runtime of matrix multiplications overlaps as much as possible with that of data movement and softmax operations, thereby fully exploiting the advantages of the dual-head pipeline and maximizing resource utilization and performance on wafer-scale chips.

F. Evaluator

The evaluator in the FACE framework is modified based on WSC-LLM [79]. It estimates the execution latency of LLM workloads on wafer-scale chips by analyzing key cost components—including computational cost, DRAM access, as well as NoC and D2D communication—and aggregates these evaluation results to derive overall performance metrics such as End-to-End (E2E) latency and throughput, which guide the selection of optimal architecture and microarchitecture configurations. The evaluator consists of two modules: intra-die evaluation and global evaluation.

In the intra-die evaluation, we refine the performance analysis models of WSC-LLM using real hardware measurement data. Specifically, the computational cost and NoC communication models are calibrated with actual data collected from a representative NPU device [85], incorporating system overheads (e.g., control costs) into performance estimation. Moreover, DRAM access is modeled based on real HBM hardware [2], further improving evaluation fidelity. These refined analysis models serve as interfaces for the global evaluation in our evaluator. In the global evaluation, we reuse WSC-LLM's D2D communication analysis model, which is based on ASTRA-sim [76], to model large-scale inter-die communication behavior.

TABLE I
CANDIDATE PARAMETERS FOR MICROARCHITECTURE AND ARCHITECTURE ON WAFER-SCALE CHIPS.

Level	Parameters		Explore Space
Micro Arch Config.	Core	SRAM (MB)	0.25, 0.5, 0.75, 1, 1.25, 1.5, 2
		Compute (GFLOPS)	26, 36, 188, 204, 276, 740, 1218, 1224, 2128, 2220, 2700, 1740, 3798, 7872
		NoC (bit)	64, 128, 256, 512, 1024
	Core array		10x10, 11x11, 12x12, 15x15, 16x16, 20x20, 22x22, 25x25, 33x33, 45x45, 49x49, 68x68
Arch Config.	Compute die (mm^2) (Case 1-10)		300, 300, 500, 500, 500, 500, 800, 800, 800, 800
	HBM chiplet (Case 1-10)		2, 4, 2, 3, 4, 6, 3, 4, 6, 8
	D2D Band. (TB/s) (Case 1-10)		2.62, 1.65, 5.02, 4.53, 4.05, 3.07, 8.13, 7.65, 6.67, 5.69
	Die array (Case 1-10)		12x9, 12x7, 9x7, 8x7, 9x6, 8x6, 7x6, 6x6, 7x5, 6x5

V. EVALUATION

A. Experiment Setup

1) Configurations for Design Space Exploration: In this work, we explore a range of configurable parameters for wafer-scale chips to demonstrate the advantages of FACE and gain deeper insights into the architecture–microarchitecture design space. To ensure a fair comparison with the wafer-scale chip used in WSC-LLM, we adopt a 12-inch wafer and choose 7nm as the default process. Table I outlines the parameter exploration space at both the microarchitecture and architecture. Notably, while focusing on the selection of compute, memory, and communication resources, the area overhead of associated control components (e.g., DMA engines and controllers) is already accounted for.

At the microarchitectural level, we primarily explore SRAM size of a core. For each SRAM configuration, we consider the overall shape and layout of the core, selecting two representative designs with minimal area fragmentation: one with higher compute capability and NoC bandwidth but a larger core area, and another with reduced compute and NoC resources in a smaller footprint. The modeling of SRAM, PE array, vector unit, NoC, and controller is informed by existing commercial hardware and accelerator designs [10], [20], [47], [48], [54], [55], [71]. At the architectural level, the primary exploration variable is the number of HBM chiplets per die, utilizing Micron HBM2E [2] with 16 GB capacity and 410 GB/s bandwidth per device. Using the optimal core configuration, we construct compute dies with varying chip areas (Small, Medium, Large). It is worth noting that the Small compute

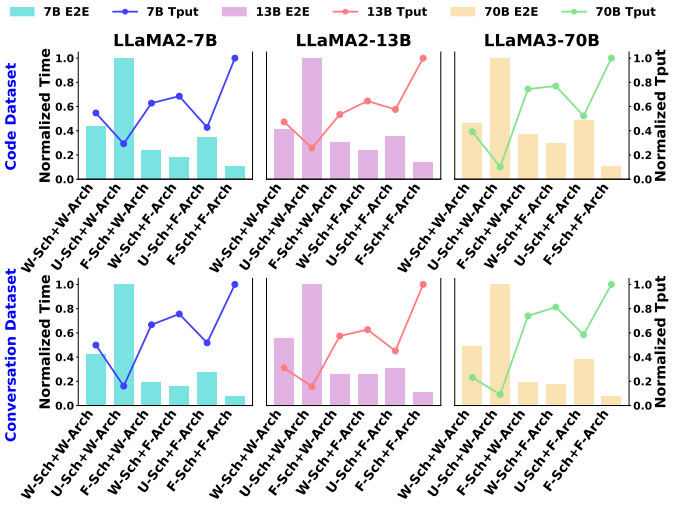

Fig. 11. Overall Comparison of Different Wafer-Scale Architectures (Arch) and LLM Scheduling Methods (Sch).

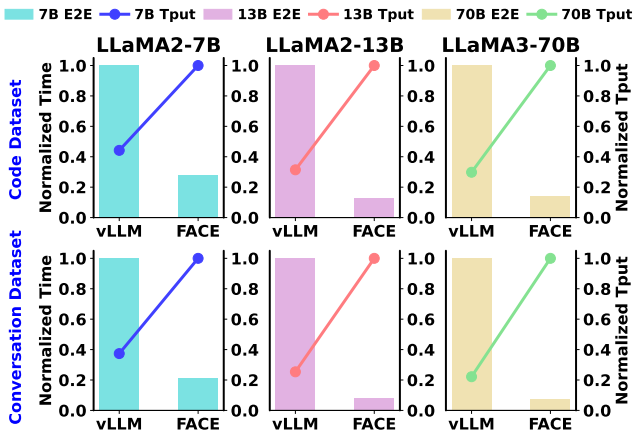

Fig. 12. Overall Comparisons of E2E Latency and Throughput between vLLM and FACE.

die offers limited feasible configurations, only allowing two cases, while Medium and Large one enable four.

2) Baselines: Our baseline consists of two key components: the baseline architecture (**Arch**) and the baseline scheduling strategy (**Sch**). For the baseline architecture, we adopt the wafer-scale chip configuration used in WSC-LLM [79], which features 54 dies per wafer. Each die provides 261.12 TFLOPS of FP16 compute power, 32.5 MB of SRAM, and 64 GB of DRAM. Both D2D and DRAM bandwidth are 2 TB/s. We denote this configuration as **W-Arch**, while the best architecture explored by our FACE framework is referred to as **F-Arch**. For the baseline scheduling strategy, we select the optimized disaggregated scheduling strategy implemented by WSC-LLM on wafer-scale chips, denoted as **W-Sch**. In addition, we implement the unified scheduling strategy, which is widely used in existing GPU-based LLM serving systems [40], [90], denoted as **U-Sch**. The fully overlapped prefill-decode scheduling strategy developed within our FACE framework is abbreviated as **F-Sch**. For example, W-Sch + W-Arch represents the baseline applying the WSC-LLM scheduling strategy on the WSC-LLM wafer-scale chip.

3) Models, Workloads, and Metrics: Given that model size is a key factor influencing performance, we select the LLaMA family [22], [69] as test LLM models, comprising LLaMA2-7B, LLaMA2-13B, and LLaMA3-70B. The latter adopts the advanced Grouped-Query Attention (GQA) mechanism. The LLaMA models are representative and widely adopted decoder-only LLMs, whose structure is consistent with other mainstream models, including GPT [11], Mistral [36], OPT [89], BLOOM [59], and DeepSeek [46]. To capture the impact of diverse workloads, we utilize real-world production traces from Azure [1], encompassing code and conversation datasets, which represent prevalent scenarios in LLM services. In the code dataset, requests arrive at an average rate of 2.57/s, with input lengths ranging from 3 to 7437 tokens and outputs from 3 to 1899 tokens. In the conversation dataset, the average arrival rate is 5.53/s, with 2–14050 input tokens and 2–1000 output tokens per request.

To comprehensively assess system performance, we adopt End-to-End (E2E) latency and throughput (Tput) as the overall evaluation metrics. E2E latency denotes the average End-to-End time per request from arrival to completion, including queuing and prefill and decode execution time. Tput is the number of requests processed per second.

B. Overall Performance

As shown in Fig. 11, compared to the baseline employing W-Sch scheduling running on the W-Arch wafer-scale architecture (W-Sch + W-Arch), our proposed F-Sch scheduling deployed on the F-Arch architecture (F-Sch + F-Arch) achieves a 3.68× reduction in E2E latency and a 1.70× improvement in throughput on average, demonstrating the effectiveness of the FACE co-exploration framework.

We observe that across all evaluated scenarios, our F-Sch scheduling consistently outperforms both W-Sch and U-Sch, regardless of whether it is deployed on W-Arch or F-Arch wafer-scale architectures. This proves that our fully overlapped prefill-decode scheduling is well suited for LLM inference on wafer-scale chips. Furthermore, in all cases, even W-Sch and U-Sch scheduling achieve better performance on the F-Arch architecture compared to W-Arch, demonstrating the architectural generality of our proposed F-Arch. This is attributed to our holistic co-exploration approach spanning wafer-scale microarchitecture and architecture levels. By optimizing from the lowest levels of hardware design, we achieve higher hardware resource integration density, which translates to enhanced LLM inference performance. Overall, ***the above results show that FACE fully unlocks the performance potential of LLM services on wafer-scale architectures*** because its prefill–decode overlapped scheduling and its architecture–microarchitecture co-exploration work in tandem to maximize the hardware resource utilization and integration.

C. Compared with GPU clusters

To demonstrate the advancement of FACE when combined with its optimized wafer-scale architecture, we also compare it against an extended vLLM [40] implementation, one of the SOTA inference serving systems on GPU clusters. For a fair

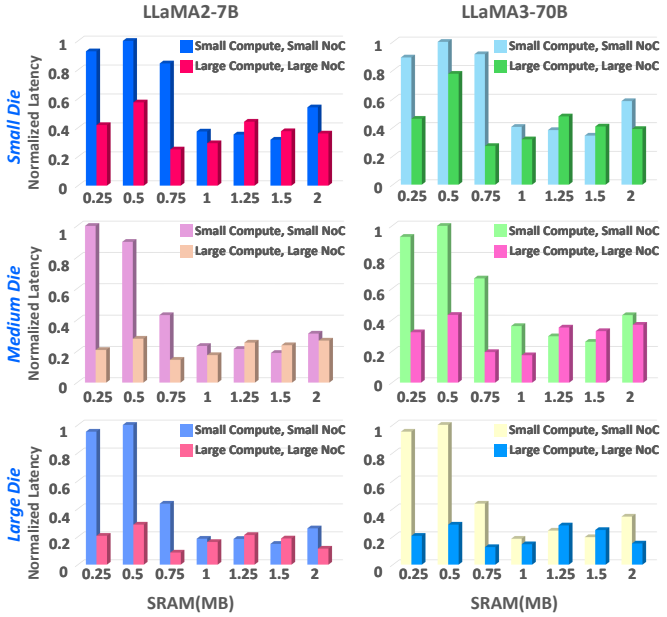

Fig. 13. Microarchitecture Exploration Based on Core Parameter Configurations on the Small, Medium and Large Compute Die.

comparison, we consider a GPU cluster that shares the same 7 nm process, comprising six NVIDIA A100 GPU [3] nodes. Within each node, the eight A100 GPUs are interconnected via NVLink [5], while inter-node communication is supported by a high-performance InfiniBand network.

Fig. 12 presents the overall performance comparison between FACE and vLLM across three LLM models and two datasets. FACE achieves a 7.23× reduction in E2E latency and a 2.29× improvement in throughput on average. Performance gains mainly arise from two factors: (1) our fully overlapped prefill–decode scheduling, which allows each instance to execute prefill and decode requests concurrently, thereby achieving substantially higher hardware utilization than vLLM (analogous to U-Sch), which processes only one request type at a time; and (2) the carefully optimized wafer-scale architecture with higher D2D and DRAM bandwidth for efficient workload balancing across instances and fast KV cache access. Furthermore, although the GPU cluster has lower D2D and DRAM bandwidth compared to our wafer-scale chip, vLLM can outperform U-Sch + F-Arch in some situations, indicating that U-Sch fails to fully exploit the hardware potential of our wafer-scale architecture. ***This highlights the necessity of designing specialized high-performance LLM scheduling strategies for wafer-scale chips.***

VI. Discussion

A. Design Space Exploration

1) Microarchitectural Granularity: Fig. 13 presents the latency results of 14 feasible wafer-scale microarchitecture configurations under two LLM models and three compute die area. For the LLaMA2-7B model, we execute LLM blocks on a 2-die instance, while the LLaMA3-70B model runs on a 4-die instance. The input LLM blocks, drawn from representative

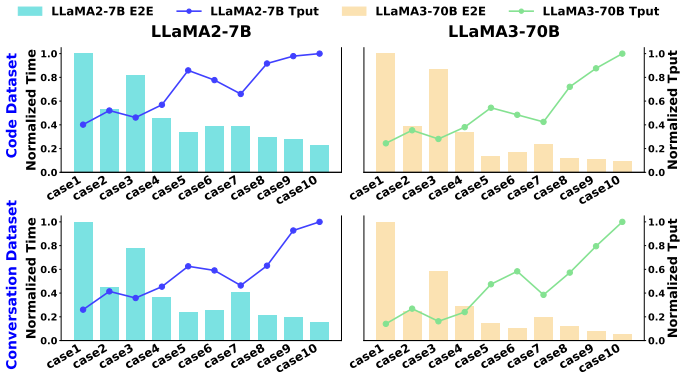

Fig. 14. Architecture Exploration across 10 Available Cases Based on the Best Microarchitecture Configuration.

code and conversation dataset workloads that enable fully prefill-decode overlap.

Experimental results demonstrate that cores with 0.75MB SRAM, high compute power, and large NoC bandwidth achieve optimal performance in most scenarios, establishing it as the best microarchitecture, while cores with 1MB SRAM achieve optimal or near-optimal performance in some situations, marking it as the second-best. *These findings suggest that moderate SRAM sizes per core enable higher hardware resource integration density and superior LLM operator performance within compute dies.* Conversely, excessively large or small SRAM sizes may reduce core count or per-core hardware resources, degrading compute die performance.

Moreover, we can also observe that, given the same SRAM size, cores equipped with higher compute capacity and larger NoC bandwidth generally deliver better performance than those with smaller compute and communication resources. This can be attributed to the fact that components other than SRAM occupy relatively little area. As a result, integrating more compute and communication units within each core only modestly reduces the number of cores per die, enabling a higher overall resource density on the compute die. ***The above observations highlight the critical design guideline for the wafer-scale microarchitecture: integrating moderate SRAM capacity together with stronger compute and NoC within each core to optimize resource integration density.***

2) Architectural Granularity: Fig. 14 presents the overall performance results of 10 feasible wafer-scale architectures (detailed in Table I) across two LLM models and two datasets using the optimal microarchitecture. These experiments cover variations in both models and workloads, providing a comprehensive and general evaluation.

Experimental results demonstrate that the wafer-scale chip (case 10) featuring large compute dies and the maximum number of HBM chiplets per die, consistently achieves the best performance across nearly all scenarios, highlighting its ability to effectively adapt to variations in model sizes and workloads. This identifies it as the optimal wafer-scale architecture. ***The above result suggests the first design guideline for the wafer-scale architecture: compute dies should be as large as possible, ideally approaching the reticle limit.***

In addition, we observe that, under a fixed compute die

area, configurations with more HBM chiplets per die generally achieve better overall performance, even though they provide lower total compute capability and reduced D2D bandwidth. This trend reflects the substantial performance benefits enabled by increased DRAM bandwidth and capacity in LLM services. However, the comparison between case 5 and case 6 shows an important exception: despite offering higher DRAM bandwidth and capacity, case 6 underperforms case 5 in most scenarios. The key reason is that case 6's lower D2D bandwidth substantially degrades both intra-instance and inter-instance communication efficiency, offsetting the benefits of its more memory resources. In this setting, the communication rather than memory becomes the dominant limitation. ***These observations lead to the second design guideline for the wafer-scale architecture: achieving high-quality LLM service requires coordinated provisioning of different hardware resources.***

B. Scalability and Future Hardware

1) Scalability: FACE features a highly flexible and configurable hardware template, as shown in Fig. 2. For wafer-scale chips with different hardware parameters, it automatically performs end-to-end scheduling for optimal LLM service. Future advances in memory (e.g., higher DRAM bandwidth) or packaging (e.g., larger D2D bandwidth, shorter die spacing) can be easily supported by tuning template parameters, allowing continued architecture-microarchitecture co-design exploration. In addition, FACE's prefill-decode overlapped scheduling strategy is also scalable across interconnect topologies. Its bidirectional ring algorithm for collective communication can seamlessly adapt to advanced switch-based fully connected networks, efficiently utilizing interconnect bandwidth. Finally, FACE can naturally extend to multi-wafer systems—when wafer-to-wafer bandwidth is limited (as OMM requires it beyond DRAM bandwidth), FACE automatically confines inter-instance workload balancing within each individual wafer.

2) Future Hardware: Recent advances in GPUs are notable, especially in large-scale interconnects such as NVL72 [5], which achieves 900 GB/s unidirectional communcation bandwidth across 72 GPUs. However, this remains insufficient for optimization strategies like OMM (Section IV-D) that demand exceeding DRAM bandwidth. Moreover, switch-based interconnects consume significantly more energy compared to wafer-scale D2D links. Fundamentally, wafer-scale chips expand the silicon area of a single device, while NVSwitch-based GPU systems interconnect multiple devices via high-speed switches.

NVIDIA GB200 employs chiplet technology to integrate two B200 GPUs [4], achieving interconnect communication bandwidth comparable to wafer-scale D2D links. Wafer-scale chips extend this chiplet-based integration from two dies to tens of dies across the entire 12-inch wafer, marking a promising direction for architectural evolution. In addition, wafer-scale chips can further scale into multi-wafer systems via switch-based interconnects, achieving the extremely large-scale and high-bandwidth hardware domain.

VII. Conclusion

This work proposes FACE, the first framework for fully overlapped prefill-decode scheduling and multi-level architecture co-exploration on wafer-scale chips. FACE exploits the fine-grained operational capabilities of wafer-scale chips to realize efficient real-time prefill-decode overlapped execution through configuration space exploration and dynamic adaptive scheduling strategies. Taking advantage of the high bandwidth of wafer-scale chips, FACE also introduces an optimized memory management approach to better accommodate the dynamic nature of LLM workloads. In terms of wafer-scale architecture and microarchitecture, we conduct a systematic analysis of the design space and perform an extensive design space exploration. Experimental results show that the architecture–microarchitecture configurations and scheduling strategy explored by FACE outperform the existing architecture design as well as the state-of-the-art LLM serving system on wafer-scale chips.

Acknowledgment

This work was supported in part by the National Science and Technology Major Project under Grant 2022ZD0115200; in part by the NSFC under Grant 62502255, Grant 62125403, Grant U24A20234, Grant 92464302 and Grant U24B20164; in part by the Beijing S&T Project Z251100008425010; in part by Shanghai Municipal Science and Technology Major Project; in part by the Natural Science Foundation of Jiangsu Province Basic Research Program under Grant BK20243042; in part by the Beijing National Research Center for Information Science and Technology; in part by the Northern IC Technology Innovation Center (Beijing) Co., Ltd under Grant QYJS20232801B; and in part by the Beijing Advanced Innovation Center for Integrated Circuits.

References

[1] "Azurepublicdataset," [Online]. Available: https://github.com/Azure/AzurePublicDataset/tree/master.

[2] "Integrating and operating hbm2e memory," [Online]. Available: https://assets.micron.com/adobe/assets/urn:aaid:aem:275edf31-79e3-4b6c-8bbd-a233babe9281/original/as/micron-hbm2e-memory-wp.pdf.

[3] "Nvidia a100," [Online]. Available: https://www.nvidia.com/en-us/data-center/a100.

[4] "Nvidia b200," [Online]. Available: https://www.nvidia.com/en-us/data-center/dgx-b200.

[5] "Nvidia gb200 nvl72: Powering the new era of computing," [Online]. Available: https://www.nvidia.com/en-us/data-center/gb200-nvl72/.

[6] "Bard, an experiment by google," [Online]. Available: https://bard.google.com/, 2023.

[7] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. Leoni, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, R. Avila, I. Babuschkin, S. Balaji, V. Balcom, P. Baltescu, H. Bao, M. Bavarian, J. Belgum, I. Bello, J. Berdine, G. Bernadett-Shapiro, C. Berner, L. Bogdonoff, O. Boiko, M. Boyd, A.-L. Brakman, G. Brockman, T. Brooks, M. Brundage, K. Button, T. Cai, R. Campbell, A. Cann, B. Carey, C. Carlson, R. Carmichael, B. Chan, C. Chang, F. Chantzis, D. Chen, S. Chen, R. Chen, J. Chen, M. Chen, B. Chess, C. Cho, C. Chu, H. Won, D. Cummings, J. Currier, Y. Dai, C. Decareaux, T. Degry, N. Deutsch, D. Deville, A. Dhar, D. Dohan, S. Dowling, S. Dunning, A. Ecoffet, A. Eleti, T. Eloundou, D. Farhi, L. Fedus, N. Felix, S. Posada, J. Forte, I. Fulford, L. Gao, E. Georges, C. Gibson, V. Goel, T. Gogineni, G. Goh, R. Gontijo-Lopes, J. Gordon, M. Grafstein, S. Gray, R. Greene, J. Gross, S. Shane, Y. Guo, C. Hallacy, J. Han, J. Harris, Y. He, M. Heaton, J. Heidecke, C. Hesse, A. Hickey, W. Hickey, P. Hoeschele,

B. Houghton, K. Hsu, S. Hu, X. Hu, J. Huizinga, S. Jain, S. Jain, J. Jang, A. Jiang, R. Jiang, H. Jin, D. Jin, S. Jomoto, B. Jonn, H. Jun, T. Kaftan, Kaiser, A. Kamali, I. Kanitscheider, N. Shirish, T. Khan, L. Kilpatrick, J. Wook, C. Kim, Y. Kim, J. Hendrik, J. Kiros, M. Knight, D. Kokotajlo, Kondraciuk, A. Kondrich, A. Konstantinidis, K. Kosic, G. Krueger, V. Kuo, M. Lampe, I. Lan, T. Lee, J. Leike, J. Leung, D. Levy, C. Ming, R. Lim, M. Lin, S. Lin, M. Litwin, T. Lopez, R. Lowe, P. Lue, A. Makanju, K. Malfacini, S. Manning, T. Markov, Y. Markovski, B. Martin, K. Mayer, A. Mayne, B. McGrew, S. Mayer, C. McLeavey, P. McMillan, J. McNeil, D. Medina, A. Mehta, J. Menick, L. Metz, A. Mishchenko, P. Mishkin, V. Monaco, E. Morikawa, D. Mossing, T. Mu, M. Murati, O. Murk, D. Mély, A. Nair, R. Nakano, R. Nayak, A. Neelakantan, R. Ngo, H. Noh, L. Ouyang, C. O'Keefe, J. Pachocki, A. Paino, J. Palermo, A. Pantuliano, G. Parascandolo, J. Parish, E. Parparita, A. Passos, M. Pavlov, A. Peng, A. Perelman, F. de, M. Petrov, H. Ponde, M. (Rai)Pokorny, M. Pokrass, V. H., T. Powell, A. Power, B. Power, E. Proehl, R. Puri, A. Radford, J. Rae, A. Ramesh, C. Raymond, F. Real, K. Rimbach, C. Ross, B. Rotsted, H. Roussez, N. Ryder, M. Saltarelli, T. Sanders, S. Santurkar, G. Sastry, H. Schmidt, D. Schnurr, J. Schulman, D. Selsam, K. Sheppard, T. Sherbakov, J. Shieh, S. Shoker, P. Shyam, S. Sidor, E. Sigler, M. Simens, J. Sitkin, K. Slama, I. Sohl, B. Sokolowsky, Y. Song, N. Staudacher, F. Petroski, N. Summers, I. Sutskever, J. Tang, N. Tezak, M. B., P. Tillet, A. Tootoonchian, E. Tseng, P. Tuggle, N. Turley, J. Tworek, J. Felipe, A. Vallone, A. Vijayvergiya, C. Voss, C. Wainwright, J. Jay, A. Wang, B. Wang, J. Ward, J. Wei, C. Weinmann, A. Welihinda, P. Welinder, J. Weng, L. Weng, M. Wiethoff, D. Willner, C. Winter, S. Wolrich, H. Wong, L. Workman, S. Wu, J. Wu, M. Wu, K. Xiao, T. Xu, S. Yoo, K. Yu, Q. Yuan, W. Zaremba, R. Zellers, C. Zhang, M. Zhang, S. Zhao, T. Zheng, J. Zhuang, W. Zhuk, and B. Zoph, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[8] D. Adiwardana, M.-T. Luong, D. R. So, J. Hall, N. Fiedel, R. Thoppilan, Z. Yang, A. Kulshreshtha, G. Nemade, Y. Lu, and Q. V. Le, "Towards a human-like open-domain chatbot," *arXiv preprint arXiv:2001.09977*, 2020.

[9] A. Agrawal, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, and R. Ramjee, "Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills," *arXiv preprint arXiv:2308.16369*, 2023.

[10] AMD, "EPYC-7003," [Online]. Available: https://www.amd.com/en/processors/epyc-7003-series.

[11] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.

[12] S. Bubeck, V. Chandrasekaran, R. Eldan, J. Gehrke, E. Horvitz, E. Kamar, P. Lee, Y. T. Lee, Y. Li, S. Lundberg, H. Nori, H. Palangi, M. T. Ribeiro, and Y. Zhang, "Sparks of artificial general intelligence: Early experiments with gpt-4," *arXiv preprint arXiv:2303.12712*, 2023.

[13] J. Cai, Z. Wu, S. Peng, Y. Wei, Z. Tan, G. Shi, M. Gao, and K. Ma, "Gemini: Mapping and architecture co-exploration for large-scale dnn chiplet accelerators," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 156–171.

[14] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. Ponde de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. Petroski Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[15] Y.-H. Chen, J. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," *ACM SIGARCH computer architecture news*, vol. 44, no. 3, pp. 367–379, 2016.

[16] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. Chung, C. Sutton, S. Gehrmann, P. Schuh, K. Shi, S. Tsvyashchenko, J. Maynez, A. Rao, P. Barnes, Y. Tay, N. Shazeer, V. Prabhakaran, E. Reif, N. Du, B. Hutchinson, R. Pope, J. Bradbury, J. Austin, M. Isard, G. Gur-Ari, P. Yin, T. Duke, A. Levskaya, S. Ghemawat, S. Dev, H. Michalewski, X. Garcia, V. Misra, K. Robinson, L. Fedus, D. Zhou, D. Ippolito, D. Luan, H. Lim, B. Zoph, A. Spiridonov, R. Sepassi, D. Dohan, S. Agrawal, M. Omernick, A. M. Dai, T. Pillai, M. Pellat, A. Lewkowycz, E. Moreira, R. Child, O. Polozov, K. Lee, Z. Zhou, X. Wang, B. Saeta, M. Diaz, O. Firat, M. Catasta, J. Wei, K. Meier-Hellstern, D. Eck, J. Dean, S. Petrov, and N. Fiedel, "Palm: Scaling language modeling with pathways," *Journal of Machine Learning Research*, vol. 24, no. 240, pp. 1–113, 2023.

[17] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," *arXiv preprint arXiv:2307.08691*, 2023.

[18] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," *Advances in neural information processing systems*, vol. 35, pp. 16 344–16 359, 2022.

[19] V.-T. Do, X.-Q. Nguyen, V.-K. Hoang, D.-H. Nguyen, S. Sabahi, J. Yang, H. Hotta, M.-T. Nguyen, and H. Le, "Automatic prompt selection for large language models," in *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, 2025, pp. 91–102.

[20] X. Dong, C. Xu, Y. Xie, and N. P. Jouppi, "Nvsim: A circuit-level performance, energy, and area model for emerging nonvolatile memory," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 31, no. 7, pp. 994–1007, 2012.

[21] G. Duan, Y. Zhang, A. Gunawan, Y. Fang, J. Mousavi, A. A. Apte, N. Ahmed, S. Sharma, S. Alur, A. Chandolu, X. Li, M. Mo, J. Jones, R. McRee, R. Limvorapitux, C. Subramani, M. Rahman, T. Ibrahim, R. Eluri, S. Kumar, S. Agraharam, and R. Manepalli, "Emib-t (tsv) advanced packaging technology emib's next evolution," in *2025 IEEE 75th Electronic Components and Technology Conference (ECTC)*. IEEE, 2025, pp. 254–258.

[22] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Sravankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Roziere, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, C. Canton, C. Nikolaidis, D. Allonsius, D. Song, D. Pintz, D. Livshits, D. Esiobu, D. Choudhary, D. Mahajan, D. Garcia-Olano, D. Perino, D. Hupkes, E. Lakomkin, E. AlBadawy, E. Lobanova, E. Dinan, E. Michael, F. Radenovic, F. Zhang, G. Synnaeve, G. Lee, G. Lewis, G. Nail, G. Mialon, G. Pang, G. Cucurell, H. Nguyen, H. Korevaar, H. Xu, H. Touvron, I. Zarov, I. Arrieta, I. Kloumann, I. Misra, I. Evtimov, J. Copet, J. Lee, J. Geffert, J. Vranes, J. Park, J. Mahadeokar, J. Shah, J. van, J. Billock, J. Hong, J. Lee, J. Fu, J. Chi, J. Huang, J. Liu, J. Wang, J. Yu, J. Bitton, J. Spisak, J. Park, J. Rocca, J. Johnstun, J. Saxe, J. Jia, K. Vasuden, K. Upasani, K. Plawiak, K. Li, K. Heafield, K. Stone, K. El-Arini, K. Iyer, K. Malik, K. Chiu, K. Bhalla, L. Rantala-Yeary, L. van, L. Chen, L. Tan, L. Jenkins, L. Martin, L. Madaan, L. Malo, L. Blecher, L. Landzaat, L. de, M. Muzzi, M. Pasupuleti, M. Singh, M. Paluri, M. Kardas, M. Oldham, M. Rita, M. Pavlova, M. Kambadur, M. Lewis, M. Si, M. Kumar, M. Hassan, N. Goyal, N. Torabi, N. Bashlykov, N. Bogoychev, N. Chatterji, O. Duchenne, O. Çelebi, P. Alrassy, P. Zhang, P. Li, P. Vasic, P. Weng, P. Bhargava, P. Dubal, P. Krishnan, P. Singh, P. Xu, Q. He, Q. Dong, R. Srinivasan, R. Ganapathy, R. Calderer, R. Silveira, R. Stojnic, R. Raileanu, R. Girdhar, R. Patel, R. Sauvestre, R. Polidoro, R. Sumbaly, R. Taylor, R. Silva, R. Hou, R. Wang, S. Hosseini, S. Chennabasappa, S. Singh, S. Bell, S. Sonia, S. Edunov, S. Nie, S. Narang, S. Raparthy, S. Shen, S. Wan, S. Bhosale, S. Zhang, S. Vandenhende, S. Batra, S. Whitman, S. Sootla, S. Collot, S. Gururangan, S. Borodinsky, T. Herman, T. Fowler, T. Sheasha, T. Georgiou, T. Scialom, T. Speckbacher, T. Mihaylov, T. Xiao, U. Karn, V. Goswami, V. Gupta, V. Ramanathan, V. Kerkez, V. Gonguet, V. Do, V. Vogeti, V. Petrovic, W. Chu, W. Xiong, W. Fu, W. Meers, X. Martinet, X. Wang, X. Ellen, X. Xie, X. Jia, X. Wang, Y. Goldschlag, Y. Gaur, Y. Babaei, Y. Wen, Y. Song, Y. Zhang, Y. Li, Y. Mao, Z. Delpierre, Z. Yan, Z. Chen, Z. Papakipos, A. Singh, A. Grattafiori, A. Jain, A. Kelsey, A. Shajnfeld, A. Gangidi, A. Victoria, A. Goldstand, A. Menon, A. Sharma, A. Boesenberg, A. Vaughan, A. Baevski, A. Feinstein, A. Kallet, A. Sangani, A. Yunus, A. Lupu, A. Alvarado, A. Caples, A. Gu, A. Ho, A. Poulton, A. Ryan, A. Ramchandani, A. Franco, A. Saraf, A. Chowdhury, A. Gabriel, A. Bharambe, A. Eisenman, A. Yazdan, B. James, B. Maurer, B. Leonhardi, B. Huang, B. Loyd, B. De, B. Paranjape, B. Liu, B. Wu,

B. Ni, B. Hancock, B. Wasti, B. Spence, B. Stojkovic, B. Gamido, B. Montalvo, C. Parker, C. Burton, C. Mejia, C. Wang, C. Kim, C. Zhou, C. Hu, C.-H. Chu, C. Cai, C. Tindal, C. Feichtenhofer, D. Civin, D. Beaty, D. Kreymer, D. Li, D. Wyatt, D. Adkins, D. Xu, D. Testuggine, D. David, D. Parikh, D. Liskovich, D. Foss, D. Wang, D. Le, D. Holland, E. Dowling, E. Jamil, E. Montgomery, E. Presani, E. Hahn, E. Wood, E. Brinkman, E. Arcaute, E. Dunbar, E. Smothers, F. Sun, F. Kreuk, F. Tian, F. Ozgenel, F. Caggioni, F. Guzmán, F. Kanayet, F. Seide, G. Medina, G. Schwarz, G. Badeer, G. Swee, G. Halpern, G. Thattai, G. Herman, G. Sizov, G. (Jack)Zhang, G. Lakshminarayanan, H. Shojanazeri, H. Zou, H. Wang, H. Zha, H. Habeeb, H. Rudolph, H. Suk, H. Aspegren, H. Goldman, I. Damlaj, I. Molybog, I. Tufanov, I.-E. Veliche, I. Gat, J. Weissman, J. Geboski, J. Kohli, J. Asher, J.-B. Gaya, J. Marcus, J. Tang, J. Chan, J. Zhen, J. Reizenstein, J. Teboul, J. Zhong, J. Jin, J. Yang, J. Cummings, J. Carvill, J. Shepard, J. McPhie, J. Torres, J. Ginsburg, J. Wang, K. Wu, K. Hou, K. Saxena, K. Prasad, K. Khandelwal, K. Zand, K. Matosich, K. Veeraraghavan, K. Michelena, K. Li, K. Huang, K. Chawla, K. Lakhotia, K. Huang, L. Chen, L. Garg, L. A, L. Silva, L. Bell, L. Zhang, L. Guo, L. Yu, L. Moshkovich, L. Wehrstedt, M. Khabsa, M. Avalani, M. Bhatt, M. Tsimpoukelli, M. Mankus, M. Hasson, M. Lennie, M. Reso, M. Groshev, M. Naumov, M. Lathi, M. Keneally, M. L., M. Valko, M. Restrepo, M. Patel, M. Vyatskov, M. Samvelyan, M. Clark, M. Macey, M. Wang, M. Jubert, M. Metanat, M. Rastegari, M. Bansal, N. Santhanam, N. Parks, N. White, N. Bawa, N. Singhal, N. Egebo, N. Usunier, N. Pavlovich, N. Dong, N. Zhang, N. Cheng, O. Chernoguz, O. Hart, O. Salpekar, O. Kalinli, P. Kent, P. Parekh, P. Saab, P. Balaji, P. Rittner, P. Bontrager, P. Roux, P. Dollar, P. Zvyagina, P. Ratanchandani, P. Yuvraj, Q. Liang, R. Alao, R. Rodriguez, R. Ayub, R. Murthy, R. Nayani, R. Mitra, R. Li, R. Hogan, R. Battey, R. Wang, R. Maheswari, R. Howes, R. Rinott, S. Jayesh, S. Datta, S. Chugh, S. Hunt, S. Dhillon, S. Sidorov, S. Pan, S. Verma, S. Yamamoto, S. Ramaswamy, S. Lindsay, S. Lindsay, S. Feng, S. Lin, S. Cindy, S. Shankar, S. Zhang, S. Zhang, S. Wang, S. Agarwal, S. Sajuyigbe, S. Chintala, S. Max, S. Chen, S. Kehoe, S. Satterfield, S. Govindaprasad, S. Gupta, S. Cho, S. Virk, S. Subramanian, S. Choudhury, S. Goldman, T. Remez, T. Glaser, T. Best, T. Kohler, T. Robinson, T. Li, T. Zhang, T. Matthews, T. Chou, T. Shaked, V. Vontimitta, V. Ajayi, V. Montanez, V. Mohan, V. Satish, V. Mangla, V. Albiero, V. Ionescu, V. Poenaru, V. Tiberiu, V. Ivanov, W. Li, W. Wang, W. Jiang, W. Bouaziz, W. Constable, X. Tang, X. Wang, X. Wu, X. Wang, X. Xia, X. Wu, X. Gao, Y. Chen, Y. Hu, Y. Jia, Y. Qi, Y. Li, Y. Zhang, Y. Zhang, Y. Adi, Y. Nam, Y. (Sid)Wang, Y. Hao, Y. Qian, Y. He, Z. Rait, Z. DeVito, Z. Rosnbrick, Z. Wen, Z. Yang, and Z. Zhao, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[23] J. Feng, Y. Huang, R. Zhang, S. Liang, M. Yan, and J. Wu, "Windserve: Efficient phase-disaggregated llm serving with stream-based dynamic scheduling," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 1283–1295.

[24] M. Gao, J. Pu, X. Yang, M. Horowitz, and C. Kozyrakis, "Tetris: Scalable and efficient neural network acceleration with 3d memory," in *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems*, 2017, pp. 751–764.

[25] H. Guo, S. Cao, L. Li, and X. Zhang, "A review on the mainstream through-silicon via etching methods," *Materials Science in Semiconductor Processing*, vol. 137, p. 106182, 2022.

[26] C. He, Y. Huang, P. Mu, Z. Miao, J. Xue, L. Ma, F. Yang, and L. Mai, "Waferllm: A wafer-scale llm inference system," *arXiv preprint arXiv:2502.04563*, 2025.

[27] K. Hegde, P.-A. Tsai, S. Huang, V. Chandra, A. Parashar, and C. W. Fletcher, "Mind mappings: enabling efficient algorithm-accelerator mapping space search," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021, pp. 943–958.

[28] C. Holmes, M. Tanaka, M. Wyatt, A. A. Awan, J. Rasley, S. Rajbhandari, R. Y. Aminabadi, H. Qin, A. Bakhtiari, L. Kurilenko, and Y. He, "Deepspeed-fastgen: High-throughput text generation for llms via mii and deepspeed-inference," *arXiv preprint arXiv:2401.08671*, 2024.

[29] S. Y. Hou, W. C. Chen, C. Hu, C. Chiu, K. C. Ting, T. S. Lin, W. H. Wei, W. C. Chiou, V. J. C. Lin, V. C. Y. Chang, C. T. Wang, C. H. Wu, and D. Yu, "Wafer-level integration of an advanced logic-memory system through the second-generation cowos technology," *IEEE Transactions on Electron Devices*, vol. 64, no. 10, pp. 4071–4077, 2017.

[30] C. Hu, H. Huang, J. Hu, J. Xu, X. Chen, T. Xie, C. Wang, S. Wang, Y. Bao, N. Sun, and Y. Shan, "Memserve: Context caching for disaggregated llm serving with elastic memory pool," *arXiv preprint arXiv:2406.17565*, 2024.

[31] C. Hu, H. Huang, L. Xu, X. Chen, J. Xu, S. Chen, H. Feng, C. Wang, S. Wang, Y. Bao, N. Sun, and Y. Shan, "Inference without interference: Disaggregate llm inference for mixed downstream workloads," *arXiv preprint arXiv:2401.11181*, 2024.

[32] Y. Hu, X. Lin, H. Wang, Z. He, X. Yu, J. Zhang, Q. Yang, Z. Xu, S. Guan, J. Fang, H. Shang, X. Tang, X. Dai, S. Wei, and S. Yin, "Wafer-scale computing: Advancements, challenges, and future perspectives," *IEEE Circuits and Systems Magazine*, vol. 24, no. 1, pp. 52–81, 2024.

[33] Y.-C. Hu, Y.-M. Liang, H.-P. Hu, C.-Y. Tan, C.-T. Shen, C.-H. Lee, and S. Hou, "Cowos architecture evolution for next generation hpc on 2.5 d system in package," in *2023 IEEE 73rd Electronic Components and Technology Conference (ECTC)*. IEEE, 2023, pp. 1022–1026.

[34] P. K. Huang, C. Y. Lu, W. H. Wei, C. Chiu, K. C. Ting, C. Hu, C. Tsai, S. Y. Hou, W. C. Chiou, C. T. Wang, and D. Yu, "Wafer level system integration of the fifth generation cowos®-s with high performance si interposer at 2500 mm2," in *2021 IEEE 71st Electronic Components and Technology Conference (ECTC)*. IEEE, 2021, pp. 101–104.

[35] Q. Huang, M. Kang, G. Dinh, T. Norell, A. Kalaiah, J. Demmel, J. Wawrzynek, and Y. S. Shao, "Cosa: Scheduling by constrained optimization for spatial accelerators," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 554–566.

[36] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, "Mistral 7b," 2023.

[37] A. K. Kamath, R. Prabhu, J. Mohan, S. Peter, R. Ramjee, and A. Panwar, "Pod-attention: Unlocking full prefill-decode overlap for faster llm inference," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2025, pp. 897–912.

[38] S.-C. Kao, G. Jeong, and T. Krishna, "Confuciux: Autonomous hardware resource assignment for dnn accelerators using reinforcement learning," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 622–636.

[39] H. Kwon, P. Chatarasi, M. Pellauer, A. Parashar, V. Sarkar, and T. Krishna, "Understanding reuse, performance, and hardware cost of dnn dataflow: A data-centric approach," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 754–768.

[40] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 611–626.

[41] J. H. Lau, G. C.-F. Chen, J. Y.-C. Huang, R. T.-S. Chou, C. C.-L. Yang, H.-N. Liu, and T.-J. Tseng, "Hybrid substrate by fan-out rdl-first panel-level packaging," *IEEE Transactions on Components, Packaging and Manufacturing Technology*, vol. 11, no. 8, pp. 1301–1309, 2021.

[42] J. H. Lau, C.-T. Ko, K.-M. Yang, C.-Y. Peng, T. Xia, P. B. Lin, J. J. Chen, P. P.-C. Huang, H.-N. Liu, T.-J. Tseng, E. Lin, and L. Chang, "Panel-level fan-out rdl-first packaging for heterogeneous integration," *IEEE Transactions on Components, Packaging and Manufacturing Technology*, vol. 10, no. 7, pp. 1125–1137, 2020.

[43] C.-r. LI, J.-h. FANG, S.-y. YIN, S.-j. WEI, and Y. HU, "Research on wafer-scale chip mapping task based on genetic algorithm," *Computer Engineering & Science*, vol. 46, no. 06, p. 993, 2024.

[44] Z. Li, L. Zheng, Y. Zhong, V. Liu, Y. Sheng, X. Jin, Y. Huang, Z. Chen, H. Zhang, J. E. Gonzalez, and S. Ion, "Alpaserve: Statistical multiplexing with model parallelism for deep learning serving," in *17th USENIX Symposium on Operating Systems Design and Implementation, OSDI 23*, 2023, pp. 663–679.

[45] S. Lie, "Cerebras architecture deep dive: First look inside the hw/sw co-design for deep learning: Cerebras systems," in *2022 IEEE Hot Chips 34 Symposium (HCS)*. IEEE Computer Society, 2022, pp. 1–34.

[46] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Zhang, H. Ding, H. Xin, H. Gao, H. Li, H. Qu, J. Cai, J. Liang, J. Guo, J. Ni, J. Li, J. Wang, J. Chen, J. Chen, J. Yuan, J. Qiu, J. Li, J. Song, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu,

L. Wang, L. Zhang, L. Xu, L. Xia, L. Zhao, L. Wang, L. Zhang, M. Li, M. Wang, M. Zhang, M. Zhang, M. Tang, M. Li, N. Tian, P. Huang, P. Wang, P. Zhang, Q. Wang, Q. Zhu, Q. Chen, Q. Du, R. Chen, R. Jin, R. Ge, R. Zhang, R. Pan, R. Wang, R. Xu, R. Zhang, R. Chen, S. Li, S. Lu, S. Zhou, S. Chen, S. Wu, S. Ye, S. Ma, S. Wang, S. Zhou, S. Yu, S. Zhou, S. Pan, T. Wang, T. Yun, T. Pei, T. Sun, W. Xiao, W. Zeng, W. Zhao, W. An, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, X. Li, X. Jin, X. Wang, X. Bi, X. Liu, X. Wang, X. Shen, X. Chen, X. Zhang, X. Chen, X. Nie, X. Sun, X. Wang, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yu, X. Song, X. Shan, X. Zhou, X. Yang, X. Li, X. Su, X. Lin, Y. Li, Y. Wang, Y. Wei, Y. Zhu, Y. Zhang, Y. Xu, Y. Huang, Y. Li, Y. Zhao, Y. Sun, Y. Li, Y. Wang, Y. Yu, Y. Zheng, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Tang, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Wu, Y. Ou, Y. Zhu, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Zha, Y. Xiong, Y. Ma, Y. Yan, Y. Luo, Y. You, Y. Liu, Y. Zhou, Z. Wu, Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Huang, Z. Zhang, Z. Xie, Z. Zhang, Z. Hao, Z. Gou, Z. Ma, Z. Yan, Z. Shao, Z. Xu, Z. Wu, Z. Zhang, Z. Li, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Gao, and Z. Pan, "Deepseek-v3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[47] S. Naffziger, N. Beck, T. Burd, K. Lepak, G. H. Loh, M. Subramony, and S. White, "Pioneering chiplet technology and design for the amd epyc™ and ryzen™ processor families : Industrial product," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 57–70.

[48] S. Naffziger, K. Lepak, M. Paraschou, and M. Subramony, "2.2 amd chiplet architecture for high-performance server and desktop products," in *2020 IEEE International Solid-State Circuits Conference (ISSCC)*, 2020, pp. 44–45.

[49] H. Oh, K. Kim, J. Kim, S. Kim, J. Lee, D.-s. Chang, and J. Seo, "Exegpt: Constraint-aware resource scheduling for llm inference," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2024, pp. 369–384.

[50] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. F. Christiano, J. Leike, and R. Lowe, "Training language models to follow instructions with human feedback," *Advances in neural information processing systems*, vol. 35, pp. 27 730–27 744, 2022.

[51] S. Pal, J. Liu, I. Alam, N. Cebry, H. Suhail, S. Bu, S. S. Iyer, S. Pamarti, R. Kumar, and P. Gupta, "Designing a 2048-chiplet, 14336-core waferscale processor," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 1183–1188.

[52] S. Pal, D. Petrisko, M. Tomei, P. Gupta, S. S. Iyer, and R. Kumar, "Architecting waferscale processors-a gpu case study," in *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2019, pp. 250–263.

[53] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient generative llm inference using phase splitting," *Power*, vol. 400, no. 700W, pp. 1–75, 2023.

[54] M. Perotti, M. Cavalcante, R. Andri, L. Cavigelli, and L. Benini, "Ara2: Exploring single-and multi-core vector processing with an efficient rvv 1.0 compliant open-source processor," *IEEE Transactions on Computers*, vol. 73, no. 7, pp. 1822–1836, 2024.

[55] N. K. Purayil, M. Perotti, T. Fischer, and L. Benini, "Araxl: A physically scalable, ultra-wide risc-v vector processor design for fast and efficient computation on long vectors," in *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 2025, pp. 1–7.

[56] R. Qin, Z. Li, W. He, M. Zhang, Y. Wu, W. Zheng, and X. Xu, "Mooncake: A kvcache-centric disaggregated architecture for llm serving," *arXiv preprint arXiv:2407.00079*, 2024.

[57] S. Roller, E. Dinan, N. Goyal, D. Ju, M. Williamson, Y. Liu, J. Xu, M. Ott, K. Shuster, E. M. Smith, Y.-L. Boureau, and W. Jason, "Recipes for building an open-domain chatbot," *arXiv preprint arXiv:2004.13637*, 2020.

[58] C. Ruan, Y. Chen, D. Tian, Y. Shi, Y. Wu, J. Li, and C. Li, "Dynaserve: Unified and elastic execution for dynamic disaggregated llm serving," *arXiv preprint arXiv:2504.09285*, 2025.

[59] T. Scao, A. Fan, C. Akiki, E. Pavlick, S. Ilić, D. Hesslow, R. Castagné, A. Luccioni, F. Yvon, M. Gallé, J. Tow, A. Rush, S. Biderman, A. Webson, P. Ammanamanchi, T. Wang, B. Sagot, N. Muennighoff, A. Moral, O. Ruwase, R. Bawden, S. Bekman, A. McMillan-Major, I. Beltagy, H. Nguyen, L. Saulnier, S. Tan, P. Suarez, V. Sanh, H. Laurençon, Y. Jernite, J. Launay, M. Mitchell, C. Raffel, A. Gokaslan, A. Simhi, A. Soroa, A. Aji, A. Alfassy, A. Rogers, A. Nitzav, C. Xu, C. Mou, C. Emezue, C. Klamm, C. Leong, D. Strien, D. Adelani, D. Radev, E. Ponferrada, E. Levkovizh, E. Kim, E. Natan, F. Toni, G. Dupont, G. Kruszewski, G. Pistilli, H. Elsahar, H. Benyamina, H. Tran, I. Yu, I. Abdulmumin, I. Johnson, I. Gonzalez-Dios, J. Rosa, J. Chim, J. Dodge, J. Zhu, J. Chang, J. Frohberg, J. Tobing, J. Bhattacharjee, K. Almubarak, K. Chen, K. Lo, L. Werra, L. Weber, L. Phan, L. allal, L. Tanguy, M. Dey, M. Muñoz, M. Masoud, M. Grandury, M. Šaško, M. Huang, M. Coavoux, M. Singh, M. Jiang, M. Vu, M. Jauhar, M. Ghaleb, N. Subramani, N. Kassner, N. Khamis, O. Nguyen, O. Espejel, O. Gibert, P. , P. Henderson, P. Colombo, P. Amuok, Q. Lhoest, R. Harliman, R. Bommasani, R. López, R. Ribeiro, S. Osei, S. Pyysalo, S. Nagel, S. Bose, S. Muhammad, S. Sharma, S. Longpre, S. Nikpoor, S. Silberberg, S. Pai, S. Zink, T. Torrent, T. Schick, T. Thrush, V. Danchev, V. Nikoulina, V. Laippala, V. Lepercq, V. Prabhu, Z. Alyafeai, Z. Talat, A. Raja, B. Heinzerling, C. Si, D. Taşar, E. Salesky, S. Mielke, W. Lee, A. Sharma, A. Santilli, A. Chaffin, A. Stiegler, D. Datta, E. Szczechla, G. Chhablani, H. Wang, H. Pandey, H. Strobelt, J. Fries, J. Rozen, L. Gao, L. Sutawika, M. Bari, M. Al-shaibani, M. Manica, N. Nayak, R. Teehan, S. Albanie, S. Shen, S. Ben-David, S. Bach, T. Kim, T. Bers, T. Fevry, T. Neeraj, U. Thakker, V. Raunak, X. Tang, Z.-X. Yong, Z. Sun, S. Brody, Y. Uri, H. Tojarieh, A. Roberts, H. Chung, J. Tae, J. Phang, O. Press, C. Li, D. Narayanan, H. Bourfoune, J. Casper, J. Rasley, M. Ryabinin, M. Mishra, M. Zhang, M. Shoeybi, M. Peyrounette, N. Patry, N. Tazi, O. Sanseviero, P. Platen, P. Cornette, P. Lavallée, R. Lacroix, S. Rajbhandari, S. Gandhi, S. Smith, S. Requena, S. Patil, T. Dettmers, A. Baruwa, A. Singh, A. Cheveleva, A.-L. Ligozat, A. Subramonian, A. Névéol, C. Lovering, D. Garrette, D. Tunuguntla, E. Reiter, E. Taktasheva, E. Voloshina, E. Bogdanov, G. Winata, H. Schoelkopf, J.-C. Kalo, J. Novikova, J. Forde, J. Clive, J. Kasai, K. Kawamura, L. Hazan, M. Carpuat, M. Clinciu, N. Kim, N. Cheng, O. Serikov, O. Antverg, O. Wal, R. Zhang, R. Zhang, S. Gehrmann, S. Mirkin, S. Pais, T. Shavrina, T. Scialom, T. Yun, T. Limisiewicz, V. Rieser, V. Protasov, V. Mikhailov, Y. Pruksachatkun, Y. Belinkov, Z. Bamberger, Z. Kasner, A. Rueda, A. Pestana, A. Feizpour, A. Khan, A. Faranak, A. Santos, A. Hevia, A. Unldreaj, A. Aghagol, A. Abdollahi, A. Tammour, A. HajiHosseini, B. Behroozi, B. Ajibade, B. Saxena, C. Ferrandis, D. McDuff, D. Contractor, D. Lansky, D. David, D. Kiela, D. Nguyen, E. Tan, E. Baylor, E. Ozoani, F. Mirza, F. Ononiwu, H. Rezanejad, H. Jones, I. Bhattacharya, I. Solaiman, I. Sedenko, I. Nejadgholi, J. Passmore, J. Seltzer, J. Sanz, L. Dutra, M. Samagaio, M. Elbadri, M. Mieskes, M. Gerchick, M. Akinlolu, M. McKenna, M. Qiu, M. Ghauri, M. Burynok, N. Abrar, N. Rajani, N. Elkott, N. Fahmy, O. Samuel, R. An, R. Kromann, R. Hao, S. Alizadeh, S. Shubber, S. Wang, S. Roy, S. Viguier, T. Le, T. Oyebade, T. Le, Y. Yang, Z. Nguyen, A. Kashyap, A. Palasciano, A. Callahan, A. Shukla, A. Miranda-Escalada, A. Singh, B. Beilharz, B. Wang, C. Brito, C. Zhou, C. Jain, C. Xu, C. Fourrier, D. Periñán, D. Molano, D. Yu, E. Manjavacas, F. Barth, F. Fuhrimann, G. Altay, G. Bayrak, G. Burns, H. Vrabec, I. Bello, I. Dash, J. Kang, J. Giorgi, J. Golde, J. Posada, K. Sivaraman, L. Bulchandani, L. Liu, L. Shinzato, M. Bykhovetz, M. Takeuchi, M. Pàmies, M. Castillo, M. Nezhurina, M. Sänger, M. Samwald, M. Cullan, M. Weinberg, M. Wolf, M. Mihaljcic, M. Liu, M. Freidank, M. Kang, N. Seelam, N. Dahlberg, N. Broad, N. Muellner, P. Fung, P. Haller, R. Chandrasekhar, R. Eisenberg, R. Martin, R. Canalli, R. Su, R. Su, S. Cahyawijaya, S. Garda, S. Deshmukh, S. Mishra, S. Kiblawi, S. Ott, S. Sang-aroonsiri, S. Kumar, S. Schweter, S. Bharati, T. Laud, T. Gigant, T. Kainuma, W. Kusa, Y. Labrak, Y. Bajaj, Y. Venkatraman, Y. Xu, Y. Xu, Y. Xu, Z. Tan, Z. Xie, Z. Ye, M. Bras, Y. Belkada, and T. Wolf, "Bloom: A 176b-parameter open-access multilingual language model," *arXiv preprint arXiv:2211.05100v4*, 2023.

[60] J. Schemmel, D. Brüderle, A. Grübl, M. Hock, K. Meier, and S. Millner, "A wafer-scale neuromorphic hardware system for large-scale neural modeling," in *2010 IEEE International Symposium on Circuits and Systems , ISCAS 2010*. IEEE, 2010, pp. 1947–1950.

[61] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, "Flashattention-3: Fast and accurate attention with asynchrony and low-precision," *Advances in Neural Information Processing Systems*, vol. 37, pp. 68 658–68 685, 2024.

[62] Y. S. Shao, J. Clemons, R. Venkatesan, B. Zimmer, M. Fojtik, N. Jiang, B. Keller, A. Klinefelter, N. Pinckney, P. Raina, S. G.Tell, Y. Zhang, J.Dally, J. Emer, C. Gray, B. Khailany, and S. W.Keckler, "Simba: Scaling deep-learning inference with multi-chip-module-based architecture," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium*

on Microarchitecture, 2019, pp. 14–27.

[63] X. Shi, C. Cai, J. Du, Z. Zhu, X. Wei, and Z. Jia, "Nexus: Taming throughput-latency tradeoff in llm serving via efficient gpu sharing," *arXiv preprint arXiv:2507.06608*, 2025.

[64] M. Song, K. Zhong, J. Zhang, Y. Hu, D. Liu, W. Zhang, J. Wang, and T. Li, "In-situ ai: Towards autonomous and incremental deep learning for iot systems," in *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2018, pp. 92–103.

[65] A. Svyatkovskiy, S. K. Deng, S. Fu, and N. Sundaresan, "Intellicode compose: Code generation using transformer," in *Proceedings of the 28th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering*, 2020, pp. 1433–1443.

[66] E. Talpes, D. Williams, and D. D. Sarma, "Dojo: The microarchitecture of tesla's exa-scale computer," in *2022 IEEE Hot Chips 34 Symposium (HCS)*. IEEE Computer Society, 2022, pp. 1–28.

[67] Z. Tan, H. Cai, R. Dong, and K. Ma, "Nn-baton: Dnn workload orchestration and chiplet granularity exploration for multichip accelerators," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 1013–1026.

[68] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[69] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. Koura, M. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. Smith, R. Subramanian, X. Ellen, B. Tang, R. Taylor, A. Williams, J. Xiang, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[70] C.-F. Tseng, C.-S. Liu, C.-H. Wu, and D. Yu, "Info (wafer level integrated fan-out) technology," in *2016 IEEE 66th Electronic Components and Technology Conference (ECTC)*. IEEE, 2016, pp. 1–6.

[71] P. Vellaisamy, H. Nair, T. Kang, Y. Ni, H. Fan, B. Qi, J. Chen, S. Blanton, and J. P. Shen, "Tempus core: Area-power efficient temporal-unary convolution core for low-precision edge dlas," in *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 2025, pp. 1–7.

[72] R. Venkatesan, Y. S. Shao, M. Wang, J. Clemons, S. Dai, M. Fojtik, B. Keller, A. Klinefelter, N. Pinckney, P. Raina, Y. Zhang, B. Zimmer, W. J. Dally, J. Emer, S. W. Keckler, and B. Khailany, "Magnet: A modular accelerator generator for neural networks," in *2019 IEEE/ACM International Conference on Computer-Aided Design, ICCAD 2019*. IEEE, 2019, pp. 1–8.

[73] H. Wang, Q. Yang, T. Wei, X. Yu, C. Li, J. Fang, G. Lu, X. Dai, L. Liu, S. Jiang, Y. Hu, S. Yin, and S. Wei, "Tmac: Training-targeted mapping and architecture co-exploration for wafer-scale chips," *Integrated Circuits and Systems*, vol. 1, no. 4, pp. 178–195, 2024.

[74] T. Wei, H. Wang, Z. Wang, S. Yin, and Y. Hu, "Spatial-aware orchestration of llm attention on waferscale chips," in *International Symposium on Advanced Parallel Processing Technologies*, 2025, pp. 386–391.

[75] W. Won, M. Elavazhagan, S. Srinivasan, S. Gupta, and T. Krishna, "Tacos: Topology-aware collective algorithm synthesizer for distributed machine learning," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 856–870.

[76] W. Won, T. Heo, S. Rashidi, S. Sridharan, S. Srinivasan, and T. Krishna, "Astra-sim2. 0: Modeling hierarchical networks and disaggregated systems for large-model training at scale," in *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2023, pp. 283–294.

[77] Q. Xiao, S. Zheng, B. Wu, P. Xu, X. Qian, and Y. Liang, "Hasco: Towards agile hardware and software co-design for tensor computation," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 1055–1068.

[78] Z. Xu, X. Dai, S. Wei, S. Yin, and Y. Hu, "Gspo: A graph substitution and parallelization joint optimization framework for dnn inference," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.

[79] Z. Xu, D. Kong, J. Liu, J. Li, J. Hou, X. Dai, C. Li, S. Wei, Y. Hu, and S. Yin, "Wsc-llm: Efficient llm service and architecture co-exploration for wafer-scale chips," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA)*, 2025, pp. 1–17.

[80] Q. Yang, J. Liu, T. Wei, Y. Jin, S. Yin, and Y. Hu, "Segmentation-aware optimization of collective for waferscale chips," in *International Symposium on Advanced Parallel Processing Technologies*, 2025, pp. 34–46.

[81] Q. Yang, T. Wei, S. Guan, C. Li, H. Shang, J. Deng, H. Wang, C. Li, L. Wang, Y. Zhang, S. Yin, and Y. Hu, "Pd constraint-aware physical/logical topology co-design for network on wafer," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA)*, 2025, pp. 49–64.

[82] X. Yang, M. Gao, Q. Liu, J. Setter, J. Pu, A. Nayak, S. Bell, K. Cao, H. Ha, P. Raina, C. Kozyrakis, and M. Horowitz, "Interstellar: Using halide's scheduling language to analyze dnn accelerators," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020, pp. 369–383.

[83] Z. Ye, L. Chen, R. Lai, W. Lin, Y. Zhang, S. Wang, T. Chen, B. Kasikci, V. Grover, A. Krishnamurthy, and L. Ceze, "Flashinfer: Efficient and customizable attention engine for llm inference serving," *arXiv preprint arXiv:2501.01005*, 2025.

[84] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for {Transformer-Based} generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 521–538.

[85] X. Yu, D. Jiang, J. Deng, J. Liu, C. Li, S. Yin, and Y. Hu, "Cramming a data center into one cabinet, a co-exploration of computing and hardware architecture of waferscale chip," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA)*, 2025, pp. 631–645.

[86] C. Zhang, L. Colagrande, R. Andri, T. Benz, G. Islamoglu, A. Nadalini, F. Conti, Y. Li, and L. Benini, "Flatattention: Dataflow and fabric collectives co-optimization for efficient multi-head attention on tile-based many-pe accelerators," *arXiv preprint arXiv:2505.18824*, 2025.

[87] H. Zhang, Y. Tang, A. Khandelwal, and I. Stoica, "Shepherd: Serving dnns in the wild," in *20th USENIX Symposium on Networked Systems Design and Implementation, NSDI 23*, 2023, pp. 787–808.

[88] M. Zhang, F. Qin, S. Chen, Y. Dai, P. Chen, and T. An, "Protrusion of through-silicon-via (tsv) copper with double annealing processes," *Journal of Electronic Materials*, vol. 51, no. 5, pp. 2433–2449, 2022.

[89] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. Lin, T. Mihaylov, M. Ott, S. Shleifer, K. Shuster, D. Simig, P. Koura, A. Sridhar, T. Wang, and L. Zettlemoyer, "Opt: Open pre-trained transformer language models," *arXiv preprint arXiv:2205.01068*, 2022.

[90] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "Sglang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems*, 2024, pp. 62 557–62 583.

[91] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving," *arXiv preprint arXiv:2401.09670*, 2024.

[92] J. Zhu, C. Xue, Y. Chen, Z. Wang, and G. Sun, "Theseus: Towards high-efficiency wafer-scale chip design space exploration for large language models," *arXiv preprint arXiv:2407.02079*, 2024.

[93] K. Zhu, Y. Gao, Y. Zhao, L. Zhao, G. Zuo, Y. Gu, D. Xie, T. Tang, Q. Xu, Z. Ye, K. Kamahori, C.-Y. Lin, Z. Wang, S. Wang, A. Krishnamurthy, and B. Kasikci, "{NanoFlow}: Towards optimal large language model serving throughput," in *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, 2025, pp. 749–765.